

SOLACE - MENTAL HEALTH ASSESSMENT PLATFORM

CLOUD COMPUTING

Submitted by

**JAYAN A (230701128)
SAUNAK RAMIYA SEBASAN (230701298)
SKANDAN KAMAL (230701322)**

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

**COMPUTER SCIENCE AND
ENGINEERING**



**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

RAJALAKSHMI ENGINEERING COLLEGE

NOVEMBER 2025

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled “**SOLACE - MENTAL HEALTH ASSESSMENT PLATFORM**” is the bonafide work of **JAYAN A(230701128)**, **SAUNAK R S(230701298)**, **SKANDAN KAMAL(230701322)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. E. M Malathy
Supervisor Head of The Department
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

SIGNATURE

Dr.S.Ponmani
Assistant Professor
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on _____

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr.MALATHY E.M, Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **DR. S. PONMANI M.E.,Ph.D.**, Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project.

ABSTRACT

Mental health awareness and accessibility remain critically limited in today's fast-paced environment, particularly when individuals require quick, private, and structured self-evaluation tools. Traditional assessment methods often demand direct professional involvement, creating barriers related to cost, availability, and privacy that prevent many users from seeking timely help. This project addresses that gap by delivering Solace, a cloud-based mental health self-assessment platform designed to be accessible, secure, and easy to use. Solace allows users to register and authenticate securely, complete structured mental health assessments based on clinically recognized screening tools such as the PHQ-9 for depression and GAD-7 for anxiety, and receive evaluated results instantly through rule-based scoring aligned with standard practices. Users may also opt to receive their results via email in the form of a generated PDF report, enabling them to retain and share findings conveniently. The platform is built using FastAPI for the backend and React for the frontend, with PostgreSQL serving as the primary relational database. Security is enforced through JWT-based authentication using the HS256 algorithm and bcrypt password hashing. The entire application is containerized using Docker and deployed on Microsoft Azure, utilizing App Service for the backend, Static Web Apps for the frontend, and Azure PostgreSQL Flexible Server for data persistence. A fully automated CI/CD pipeline built with GitHub Actions ensures consistent, reliable deployments with minimal manual intervention. The expected outcomes of this project include API response times below 150 milliseconds, secure and scalable cloud infrastructure, broad accessibility for the general public, and a fully automated deployment pipeline. By combining modern web technologies with cloud-native deployment practices, Solace demonstrates how mental health tooling can be made more approachable, private, and efficient.

TABLE OF CONTENTS

Chapter No.	Title	Page No .
1	Introduction	7
1.1	Problem Statement	7
1.2	Objective of the Project	7
1.3	Scope and Boundaries	8
1.4	Stakeholders and End Users	8
1.5	Technologies Used	9
1.6	Organization of the Report	11
2	System Design and Architecture	12
2.1	Requirement Summary	12
2.2	Proposed Solution Overview	14
2.3	Cloud Deployment Strategy	15
2.4	Infrastructure Requirements	16
2.5	Azure Services Mapping and Justification	18
3	Devops Implementation	19
3.1	CI / CD Strategy	19
3.2	Containerization Strategy	23
3.3	Deployment Workflow	24
4	Cloud Operations and Security	27
4.1	DevSecops Integration	27
4.2	Monitoring and observability	28
4.3	Access Control	29
4.4	Backup & Disaster Recovery	30

5	Results and Discussion	31
5.1	Implementation Summary	31
5.2	Challenges Faced and Resolutions	32
5.3	Performance and Cost Optimization	33
5.4	Key Learning and Team Contribution	36
6	Conclusion and Future Work	37
6.1	Conclusion	37
6.2	Future Works	38
	Appendices	41
	Result	44

CHAPTER 1 - INTRODUCTION

1.1 PROBLEM STATEMENT

Mental health awareness and access to self-evaluation tools remain a significant challenge for a large portion of the general public. Many individuals who experience symptoms of anxiety, depression, or other mental health conditions are either unaware of standardized screening methods or face practical barriers that prevent them from seeking timely evaluation. Traditional mental health assessments typically require scheduling appointments with licensed professionals, which introduces constraints related to cost, availability, geographic accessibility, and social stigma. These barriers collectively discourage individuals from taking the first step toward understanding their own mental health.

The scale of this problem is considerable. A significant portion of individuals experiencing mental health difficulties never receive any form of evaluation or support, largely because the entry point to care feels inaccessible or intimidating. For many, the idea of visiting a clinic or disclosing personal struggles to a professional at an early stage feels like too large a step. What is often needed is a low-friction, private, and reliable way to first understand whether one's symptoms align with recognized patterns of conditions such as depression or anxiety.

Furthermore, existing digital alternatives often lack the privacy assurances and clinical grounding that users need to trust the platform with sensitive personal information. Many general-purpose wellness applications do not use validated screening instruments, making their outputs unreliable for any meaningful self-awareness. There is therefore a clear and pressing need for a self-service, privacy-respecting platform that provides structured, clinically aligned assessments without requiring professional involvement at the point of evaluation. Solace is designed specifically to address this need.

1.2 OBJECTIVES OF THE PROJECT:

The primary objective of this project is to design and deploy a secure, cloud-hosted mental health self-assessment platform that is accessible to any user without barriers. The platform aims to bridge the gap between the need for mental health awareness and the availability of reliable, private, and easy-to-use digital tools. The specific objectives are as follows.

The first objective is to build a secure user authentication system that protects personal data through industry-standard encryption and token-based access control, ensuring that user information remains confidential at all times. The second objective is to provide accurate rule-based mental health assessments using clinically recognized screening tools such as the PHQ-9 for depression screening and the GAD-7 for anxiety screening, ensuring that the results delivered to users are meaningful, interpretable, and aligned with standard clinical scoring practices. The third objective is to deploy a scalable and reliable cloud infrastructure on Microsoft Azure that guarantees consistent availability and performance regardless of user load. The fourth objective is to enable a fully automated CI/CD pipeline using GitHub Actions, which streamlines the build, testing, and deployment process and reduces the risk of human error during releases. The fifth objective is to ensure fast API response times consistently below 150 milliseconds, delivering a smooth and responsive user experience that encourages engagement with the platform.

1.3 SCOPE AND BOUNDARIES:

The scope of this project covers the complete design, development, and deployment of a full-stack web application with integrated cloud infrastructure and automated deployment pipelines. On the functional side, the platform includes user registration and login with secure credential management, structured mental health assessments based on the PHQ-9 and GAD-7 questionnaires, rule-based result evaluation and scoring, email-based result delivery, and PDF report generation for user records. The backend is fully containerized using Docker and deployed on Azure App Service, with persistent data stored in Azure PostgreSQL Flexible Server. The frontend is hosted on Azure Static Web Apps, and a complete CI/CD pipeline automates the entire build and deployment workflow through GitHub Actions.

The system is designed to be modular and maintainable, with clear separation between the frontend and backend layers. This separation ensures that each component can be updated, scaled, or replaced independently without disrupting the overall platform. Infrastructure provisioning and deployment are handled through configuration-driven workflows, reducing manual overhead and improving reproducibility across environments.

The current scope deliberately excludes certain features to maintain focus and ensure project feasibility within the given timeline. AI or machine learning based analysis of assessment responses is not included in this version, as the project relies on deterministic rule-based scoring derived from established clinical guidelines. A conversational chatbot interface for guided mental health support is also outside the present scope. Additionally, longitudinal tracking of user assessment results over time, which would allow users to monitor changes in their mental health status across multiple sessions, has been identified as a valuable feature but is reserved for future development.

1.4 STAKEHOLDERS AND END USERS:

The primary end users of the Solace platform are members of the general public who seek a quick, private, and accessible means of evaluating their mental health status. The platform is intentionally user-agnostic, meaning it does not target any specific age group, profession, or demographic. It is designed to be approachable and usable by anyone with a basic level of internet access and digital literacy, requiring no prior knowledge of mental health terminology or clinical processes. The simplicity of the interface and the clarity of the assessment flow ensure that users can complete the process without confusion or technical difficulty.

From a technical standpoint, the developer serves as the sole stakeholder responsible for the design, implementation, testing, and ongoing maintenance of the platform. All architectural decisions, technology selections, and deployment configurations are managed by the development team. Microsoft Azure acts as the infrastructure provider, supplying the managed cloud services on which the application is hosted, including compute, database, storage, and networking capabilities. The reliability and availability of the platform are therefore partly dependent on the service level commitments provided by Azure.

1.5 TECHNOLOGIES USED:

The project incorporates a carefully selected technology stack designed for performance, security, and ease of deployment. Each technology was chosen based on its suitability for the specific layer of the application it serves, its community support, and its compatibility with the overall cloud-native architecture of the platform.

Table 1: Backend Technologies

Technology	Purpose
FastAPI	Primary web framework for building RESTful APIs
Python 3.10+	Core programming language
SQLModel	ORM library for database interaction and schema management
PostgreSQL / SQLite	Relational database for structured data storage

uv	Python project and dependency manager
ReportLab	PDF generation for assessment result reports
SMTP	Email delivery service for result sharing

Table 2: Frontend Technologies

Technology	Purpose
React (Vite)	Frontend user interface framework
Bun	JavaScript package manager
Nginx	Reverse proxy for serving the frontend

Table 3: DevOps and Cloud Infrastructure

Technology	Purpose
Docker	Containerization of the backend application
GitHub Actions	CI/CD pipeline automation

Azure App Service	Cloud hosting for the containerized backend
Azure Static Web Apps	Hosting for the React frontend
Azure PostgreSQL Flexible Server	Managed relational database service
Azure Container Registry	Storage and distribution of Docker images

The backend is built entirely in Python using FastAPI, which provides high-performance asynchronous request handling and automatic API documentation generation. SQLAlchemy serves as the ORM layer, combining the strengths of SQLAlchemy and Pydantic to enable clean data modeling and validation. The frontend is developed in React using the Vite build tool, which offers significantly faster development builds compared to traditional bundlers. Bun is used as the package manager for its speed advantages over npm and yarn. Nginx acts as a reverse proxy in front of the React application, handling routing and serving static assets efficiently. On the infrastructure side, Docker ensures that the backend application runs consistently across development and production environments. GitHub Actions automates the entire pipeline from code commit to cloud deployment, and Azure provides the managed services necessary to run the application at scale without the overhead of managing raw virtual machines.

1.6 ORGANIZATION OF THE REPORT

This report is structured to provide a complete and logical account of the project from initial conception through to final deployment and evaluation. Each chapter builds upon the previous one, progressively covering the motivation behind the project, the design decisions made, the implementation details, the operational setup, and the outcomes achieved.

Chapter 1 introduces the problem context, project objectives, scope and boundaries, stakeholders, technologies used, and the overall structure of the report. It establishes the foundation for understanding why the project was undertaken and what it aims to achieve. Chapter 2 presents the system design and architecture in detail, covering both functional and non-functional requirements, the proposed solution overview, the cloud deployment strategy, infrastructure specifications, and a mapping of Azure services to project requirements along with cost justification. Chapter 3 details the DevOps implementation, including the CI/CD pipeline configuration using GitHub Actions, the Docker containerization strategy, deployment workflows for both the backend and frontend, and the GitHub secrets management approach. Chapter 4 addresses cloud operations and security, discussing the authentication and authorization mechanisms, access control policies, monitoring and logging setup through Azure services, and the backup and disaster recovery provisions offered by the managed infrastructure. Chapter 5 presents the results of the implementation, including

performance benchmarks, cost observations over the deployment period, challenges encountered during development and deployment, and the specific resolutions that were applied to overcome them. Chapter 6 concludes the report with a summary of the achievements delivered by the project and a detailed roadmap of planned future enhancements that would extend the platform's capabilities. The document closes with references and appendices containing pipeline configuration files, infrastructure screenshots, and system deployment

CHAPTER 2 – SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

Functional Requirements

The Solace platform is designed around a set of core functional requirements that define the capabilities the system must deliver to its users. The first and most fundamental requirement is user registration and login. New users must be able to create an account using a unique identifier and a password, and returning users must be able to authenticate securely to access their personalized session. This forms the entry point for all other platform interactions and must be handled with strict security controls to protect user credentials. The registration process includes basic input validation to ensure that duplicate accounts are not created and that passwords meet minimum security standards before being stored.

The second functional requirement is token-based authentication. Once a user successfully logs in, the system must issue a JSON Web Token that is used to authorize all subsequent requests made by that user. This token-based approach eliminates the need for server-side session storage and enables stateless, scalable API communication between the frontend and backend. The token is signed using the HS256 algorithm and carries a user identifier claim that is verified by middleware on every protected route before the request is processed further.

The third requirement is assessment submission. Authenticated users must be able to access and complete structured mental health assessments based on the PHQ-9 and GAD-7 questionnaires. The PHQ-9, or Patient Health Questionnaire-9, is a nine-item instrument used to screen for the presence and severity of depression. The GAD-7, or Generalized Anxiety Disorder-7, is a seven-item instrument used to assess the severity of generalized anxiety disorder. The platform must present these questions in a clear and accessible format and accept the user's responses for processing without ambiguity.

The fourth requirement is result calculation. Upon submission of an assessment, the system must evaluate the responses using rule-based scoring logic derived from the clinical guidelines

associated with the PHQ-9 and GAD-7 instruments. For the PHQ-9, scores are summed and classified into severity bands ranging from minimal to severe depression. For the GAD-7, a similar summation and classification approach is applied, with severity levels ranging from minimal to severe anxiety. The calculated score and its corresponding severity classification must be returned to the user immediately upon completion, presented in a clear and understandable format.

The fifth requirement is email delivery. Users must have the option to receive their assessment results via email. The system must generate a structured PDF report of the results using ReportLab and deliver it to the user's registered email address through an SMTP-based email service. The PDF report must include the assessment type, individual responses, total score, and severity classification, giving the user a complete and portable record of their evaluation.

Non-Functional Requirements

Beyond the core functional capabilities, the system is held to a set of non-functional requirements that govern its performance, security, scalability, and reliability. These requirements define the quality standards the platform must meet in order to be considered production-ready and trustworthy for use by members of the general public handling sensitive personal health information.

In terms of performance, the platform targets API response times consistently below 150 milliseconds for standard operations such as login, assessment submission, and result retrieval. This ensures that users experience a fast and fluid interaction with the platform regardless of the operation being performed. The asynchronous request handling capabilities of FastAPI contribute significantly to meeting this target, as they allow the server to process multiple concurrent requests without blocking.

Security is enforced at multiple levels throughout the application stack. Passwords are stored exclusively as bcrypt hashes, ensuring that plaintext credentials are never persisted in the database even in the event of a data breach. Authentication tokens are signed using the HS256 algorithm with a securely stored secret key that is supplied through environment variables and never exposed in the source code. All tokens are subject to expiration to limit the window of exposure in the event of a compromised token. API routes that handle sensitive operations are protected through OAuth2 bearer token middleware and require a valid, unexpired token before processing any request. Database connections are established over SSL to prevent credential interception in transit.

Scalability is addressed through cloud-based deployment on Microsoft Azure. By hosting the backend on Azure App Service and the database on Azure PostgreSQL Flexible Server, the platform inherits the scaling capabilities of managed cloud infrastructure. The containerized nature of the backend further ensures that the application can be redeployed or scaled horizontally with minimal

configuration changes, as the Docker image encapsulates all runtime dependencies and configuration.

Reliability is supported through the use of automated CI/CD pipelines. Every code change pushed to the repository triggers an automated build and deployment process, reducing the risk of manual deployment errors and ensuring that the production environment consistently reflects the latest validated state of the codebase. Azure App Service provides automatic container restart behaviour in the event of application crashes, further contributing to the overall reliability of the deployed platform.

2.2 PROPOSED SOLUTION OVERVIEW

Solace is designed as a logically separated full-stack web application with a monolithic FastAPI backend and a React-based frontend, connected through a RESTful API interface. The decision to use a monolithic backend architecture was deliberate, given the scope and scale of the project. A single, well-structured FastAPI application is sufficient to handle all business logic, authentication, assessment processing, result calculation, email delivery, and database interactions without the operational overhead that a microservices architecture would introduce at this scale. The monolithic approach also simplifies debugging, testing, and deployment, making it the appropriate choice for a project maintained by a small development team.

The frontend communicates with the backend exclusively through HTTP API calls, with all protected routes requiring a valid JWT bearer token in the request header. This clean separation between presentation and business logic ensures that each layer can be developed, tested, and deployed independently without coupling. The React frontend is built using Vite, which offers significantly faster build times compared to traditional bundlers and provides a superior local development experience. The compiled static assets are served through Azure Static Web Apps, which provides global content delivery and eliminates the need for a dedicated web server to serve frontend assets in production. Nginx is used as a reverse proxy within the containerized development environment to handle routing efficiently.

On the backend, FastAPI handles incoming requests and delegates operations to the appropriate service layers within the application. The routing layer receives HTTP requests, validates incoming data using Pydantic models, and invokes the relevant business logic functions. SQLAlchemy manages all database interactions, providing a clean and type-safe ORM interface that combines the expressiveness of SQLAlchemy with the data validation capabilities of Pydantic. User data, assessment definitions, and result records are all persisted in Azure PostgreSQL Flexible Server, which provides automated backups, high availability configuration, and managed maintenance windows.

When a user completes an assessment, the scoring engine within the backend applies the rule-based logic corresponding to the PHQ-9 or GAD-7 instrument. For the PHQ-9, the engine sums the scores across all nine items and maps the total to one of five severity classifications: minimal, mild, moderate, moderately severe, or severe. For the GAD-7, the same summation approach is applied with four severity levels: minimal, mild, moderate, and severe. The calculated result is returned immediately in the API response. If the user opts for email delivery, the backend generates a formatted PDF report using ReportLab, incorporating the assessment type, individual item scores, total score, and severity classification, before dispatching it to the user's email address via SMTP.

The entire backend is packaged as a Docker container image and stored in Azure Container Registry. Deployment to Azure App Service is triggered automatically through the GitHub Actions CI/CD pipeline upon every push to the main branch, ensuring that the production environment is always current and that every deployed version corresponds to a specific, traceable commit in the source repository.

2.3 CLOUD DEPLOYMENT STRATEGY

The deployment strategy for Solace is built entirely on Microsoft Azure, leveraging a combination of Platform-as-a-Service offerings to minimize infrastructure management overhead while maintaining enterprise-grade reliability and performance. Azure was selected as the cloud provider due to its comprehensive managed service offerings, the availability of a student subscription with sufficient credits for the project duration, strong support for containerized workloads, and the geographic availability of data centers in the South Asia region, which reduces network latency for the target user base.

The backend application is deployed as a Docker container on Azure App Service, which provides a fully managed hosting environment for containerized web applications. App Service handles the underlying compute infrastructure, operating system patching, and runtime management, allowing the development effort to remain focused entirely on application logic rather than server administration. The container image is pulled directly from Azure Container Registry during deployment, ensuring a secure and private image distribution channel that is not exposed to public registries.

The frontend is deployed to Azure Static Web Apps, which provides a globally distributed content delivery network for static assets. This service offers built-in SSL certificate provisioning, custom domain support, and seamless integration with GitHub Actions for automated deployment through a dedicated deployment token. By serving the frontend through a CDN, the platform ensures fast and consistent load times for users across different geographic locations. The Static Web Apps service also handles routing configuration for single-page applications, ensuring that client-side navigation works correctly without requiring server-side routing support.

The database is hosted on Azure PostgreSQL Flexible Server, deployed in the Central India region to minimize latency between the database and the App Service backend, which is hosted in the South India region. This managed service handles routine database administration tasks such as automated backups, security patching, and high availability configuration, significantly reducing the operational burden on the development team. The backend application connects to the database over a secure SSL-enforced connection, with the connection string supplied through environment variables configured in the App Service deployment settings.

Azure Container Registry serves as the private repository for all Docker images used by the platform. Images are built and pushed to the registry as part of the CI/CD pipeline using credentials stored as GitHub repository secrets. The App Service deployment is then updated to reference the newly pushed image tag, completing the end-to-end automated deployment flow. This approach ensures that only verified, pipeline-built images originating from the authenticated source repository are ever deployed to the production environment, maintaining a clear and auditable deployment chain.

2.4 INFRASTRUCTURE REQUIREMENTS

The infrastructure supporting the Solace platform is provisioned across multiple Azure services, each configured to meet the specific demands of its respective workload while remaining within the constraints of the Azure Student Subscription budget.

The backend compute environment is provided by Azure App Service, configured to run the Docker container pulled from Azure Container Registry. The App Service plan is sized to accommodate the expected request load while remaining cost-efficient. The service is configured to restart automatically in the event of a container failure, providing a basic level of self-healing behaviour without requiring manual intervention. Environment variables including the database connection string, JWT secret key, SMTP credentials, and other sensitive configuration values are supplied directly through the App Service configuration settings, ensuring they are available to the running container without being embedded in the Docker image or the source code repository.

The frontend infrastructure requires minimal configuration beyond the initial Static Web App provisioning. The deployment workflow uploads the compiled React build output to the Static Web Apps service, which handles asset distribution, routing, and SSL automatically. A `staticwebapp.config.json` file is included in the frontend repository to configure URL rewriting rules that ensure all navigation requests are correctly handled by the React router rather than returning 404 errors from the CDN.

Database Configuration

Parameter	Value
Service	Azure PostgreSQL Flexible Server
Compute Tier	Burstable
SKU	Standard_B1ms
vCores	1
RAM	2 GB
Storage	32 GB
Autogrow	Disabled
Backup Retention	Managed by Azure
SSL Mode	Required
Region	Central India

The database is configured using the Burstable compute tier with the Standard_B1ms SKU, which provides one virtual core and two gigabytes of RAM. This configuration is well suited to the expected workload, where database operations consist primarily of short-duration reads and writes associated with user authentication, assessment submission, and result retrieval. The burstable tier allows the database to handle brief periods of elevated load by consuming accumulated CPU credits, making it a cost-effective choice for workloads that do not require sustained high CPU utilization. Storage is provisioned at 32 gigabytes, which is more than sufficient for the volume of user and assessment data expected throughout the project lifecycle. Autogrow is disabled to maintain predictable and controlled storage costs under the student subscription. SSL mode is set to required for all connections, ensuring that data transmitted between the application and the database is encrypted in transit.

2.5 AZURE SERVICES MAPPING AND JUSTIFICATION

The following table provides a consolidated mapping of each project requirement to the corresponding Azure service selected to fulfill it, along with the configuration tier, purpose, and cost justification for each service.

Table 4: Azure Service Mapping

Requirement	Azure Service	Tier	Purpose	Justification	Est. Cost for 2.5 weeks
Backend Hosting	Azure App Service	Basic B1	Host containerized FastAPI backend	Managed hosting with container support, auto-restart, and environment variable configuration	\$0.78
Frontend Hosting	Azure Static Web Apps	Free	Serve React frontend globally	Free tier with built-in CDN, SSL, and GitHub Actions integration	Free
Database	Azure PostgreSQL Flexible Server	Burstable B1ms	Store user, assessment, and result data	Fully managed with automated backups, SSL enforcement, and high availability options	\$5.98

Image Registry	Azure Container Registry	Basic	Store and distribute Docker images	Private registry with secure access, integrated with App Service pull-on-deploy	\$3.12
Resource Management	Azure Resource Group	N/A	Logical grouping of all Azure resources	Simplifies management, access control, and consolidated billing across all services	Free

All services are provisioned under an Azure Student Subscription, which provides sufficient credit to cover the operational costs of the platform throughout the development and evaluation period. The total estimated cost for the two and a half week deployment period amounts to approximately \$9.88, clearly demonstrating the cost efficiency of the selected architecture.

The choice to use managed Platform-as-a-Service offerings across all layers of the infrastructure is a deliberate architectural decision that prioritizes developer productivity and operational simplicity. Rather than provisioning and managing raw virtual machines, configuring operating systems, and handling infrastructure-level concerns manually, the team is able to direct all effort toward application development and quality. Each Azure service selected abstracts away a specific layer of infrastructure complexity while providing the reliability, security, and scalability guarantees required for a production-grade deployment. This approach also ensures that the platform remains maintainable by a single developer without requiring specialized expertise in server administration, database management, or network configuration.

CHAPTER 3 – DEVOPS IMPLEMENTATION

3.1 CONTINUOUS INTEGRATION AND DEPLOYMENT STRATEGY

The Solace platform employs a fully automated CI/CD pipeline built with GitHub Actions to manage the build, testing, and deployment of both the backend and frontend components. This pipeline eliminates the need for manual deployment steps, reduces the risk of human error during releases, and ensures that the production environment consistently reflects the latest validated state of the codebase. Every push to the main branch triggers the pipeline automatically, initiating a sequential workflow that builds the backend container image, pushes it to Azure Container Registry, deploys it to Azure App Service, and subsequently builds and deploys the frontend to Azure Static Web Apps.

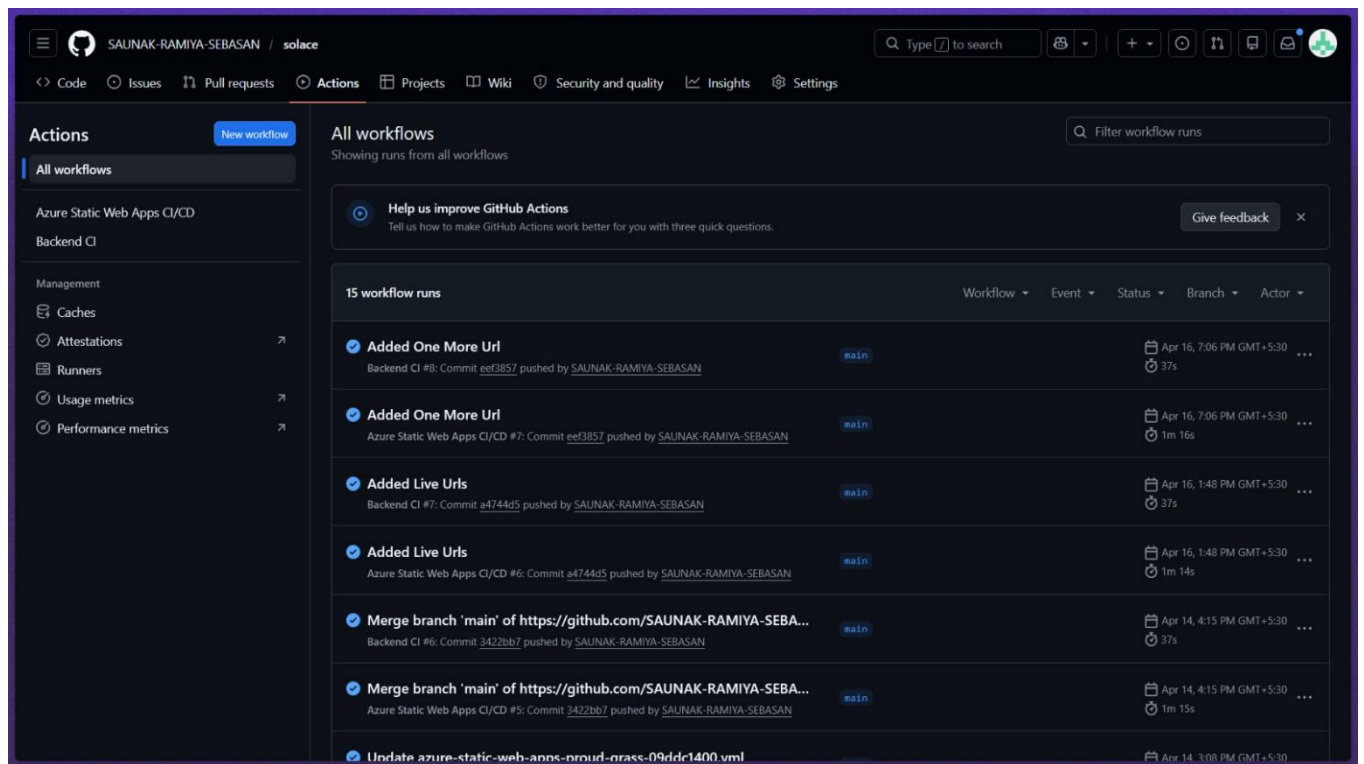


Fig. 1. Github Actions Workflow

The decision to use GitHub Actions as the CI/CD platform was driven by its tight integration with the GitHub source repository, its support for secret management, its extensive library of community-maintained actions for Azure deployments, and its availability at no additional cost under the student subscription. The pipeline configuration is defined entirely in YAML workflow files stored within the repository, making the deployment process transparent, version-controlled, and reproducible.

The pipeline is structured as a two-stage sequential workflow. The first stage handles the backend build and deployment, and the second stage handles the frontend build and deployment. The frontend deployment stage is configured to depend on the successful completion of the backend stage, ensuring that the frontend is only updated after the backend it communicates with has been successfully deployed and is available. This dependency ordering prevents scenarios where a newly deployed frontend attempts to communicate with a backend that is still being updated or is temporarily unavailable.

Pipeline Stage 1: Backend Build and Deployment

The backend deployment stage begins by checking out the latest source code from the repository. It then authenticates with Docker Hub and Azure Container Registry using credentials stored as GitHub repository secrets. A unique version tag is generated for the container image using the current date and the short SHA of the triggering commit,

following the format vYYYYMMDD-shortsha. This versioning strategy ensures that every deployed image is uniquely identifiable and traceable to a specific commit, enabling straightforward rollback to any previous version if a regression is introduced.

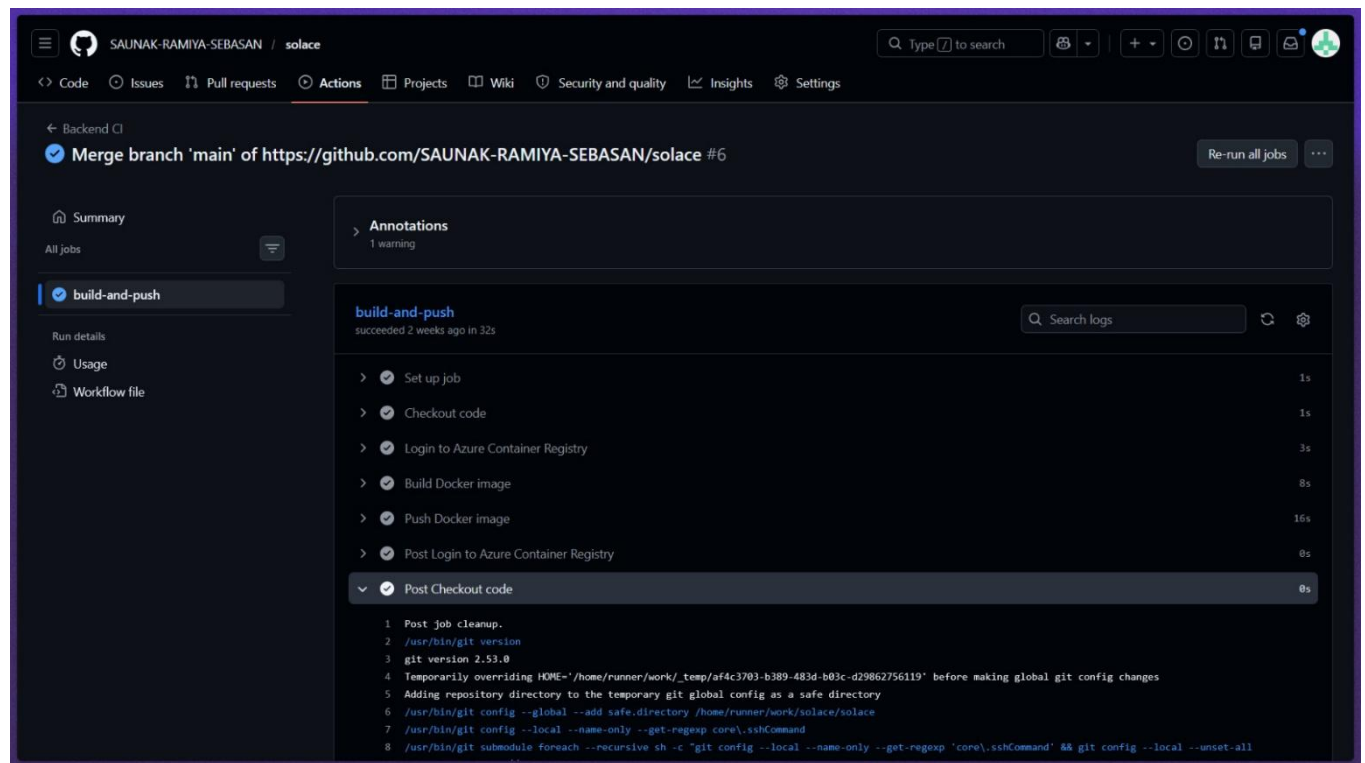


Fig. 2. Backend Deployment Job Run

The Docker image is built from the backend Dockerfile using a multi-stage build process and pushed to Azure Container Registry under both the generated version tag and the latest tag. The version tag is used for the actual App Service deployment to ensure immutability and traceability, while the latest tag is maintained for convenience during local development and testing.

Following the image push, the pipeline authenticates with Azure using a Service Principal whose credentials are stored as GitHub secrets. The Azure App Service deployment is then updated to reference the newly pushed container image, triggering a rolling update that replaces the running container with the new version. Azure App Service handles the container pull and restart process, and the deployment is considered complete once the new container is confirmed to be running and healthy.

Pipeline Stage 2: Frontend Build and Deployment

The frontend deployment stage begins after the backend stage has completed successfully. It checks out the source code, sets up the Node.js environment, and installs dependencies using the Bun package manager. The backend API base URL, which is required by the React application to

communicate with the deployed backend, is supplied to the build process as an environment variable sourced from the VITE_API_BASE GitHub secret. This approach ensures that the compiled frontend assets always reference the correct backend endpoint for the environment they are being deployed to.

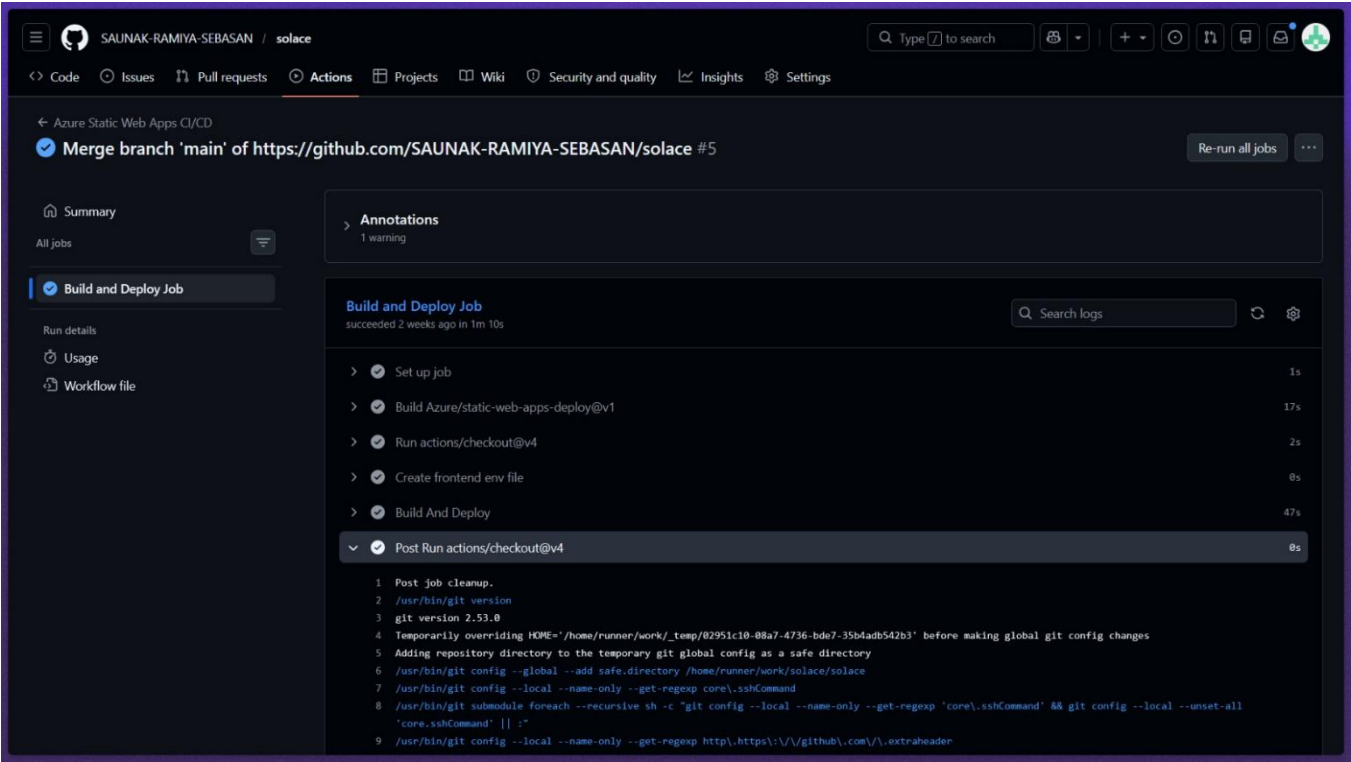


Fig. 3. Frontend Deployment Job Run

The React application is compiled into a set of optimized static assets using the Vite build tool. The compiled output is then deployed to Azure Static Web Apps using the official Azure Static Web Apps deployment action, authenticated through the AZURE_STATIC_WEB_APPS_API_TOKEN secret. The Static Web Apps service receives the build output and distributes it across its global CDN, making the updated frontend immediately available to users worldwide.

GitHub Secrets Configuration

The pipeline relies on a set of repository secrets to supply sensitive configuration values without exposing them in the workflow files or the source code. The following secrets are required for the pipeline to function correctly.

Table 5: GitHub Secrets

Secret Name	Purpose
ACR_LOGIN_SERVER	Azure Container Registry login server URL
ACR_USERNAME	Azure Container Registry authentication username
ACR_PASSWORD	Azure Container Registry authentication password
AZURE_STATIC_WEB_APPS_API_TOKEN	Deployment token for Azure Static Web Apps
VITE_API_BASE	Backend API base URL injected into the frontend build

All secrets are stored encrypted within the GitHub repository settings and are only exposed to the workflow runner at execution time. They are never printed to workflow logs or accessible outside of the pipeline execution context, ensuring that sensitive credentials remain protected throughout the deployment process.

3.2 CONTAINERIZATION STRATEGY

The backend application is packaged as a Docker container image to ensure consistent, reproducible deployments across all environments from local development through to production. Containerization eliminates the class of deployment failures caused by environment-specific differences in runtime versions, installed dependencies, or system configuration, providing a guarantee that the application behaves identically regardless of where the container is executed.

The Docker image is built using a multi-stage build process defined in the backend Dockerfile. The multi-stage approach separates the build environment from the runtime environment, ensuring that the final image contains only the artifacts necessary to run the application and none of the build tools, intermediate files, or development dependencies that are required during the compilation phase. This results in a smaller, more secure final image with a reduced attack surface compared to a single-stage build.

The first stage of the build process uses a Python base image to install the project dependencies defined in the requirements file. The uv package manager is used for dependency installation due to its significantly faster resolution and installation times compared to pip. Dependencies are installed into a virtual environment within the container to maintain isolation from the system Python installation. The application source code is then copied into the build stage.

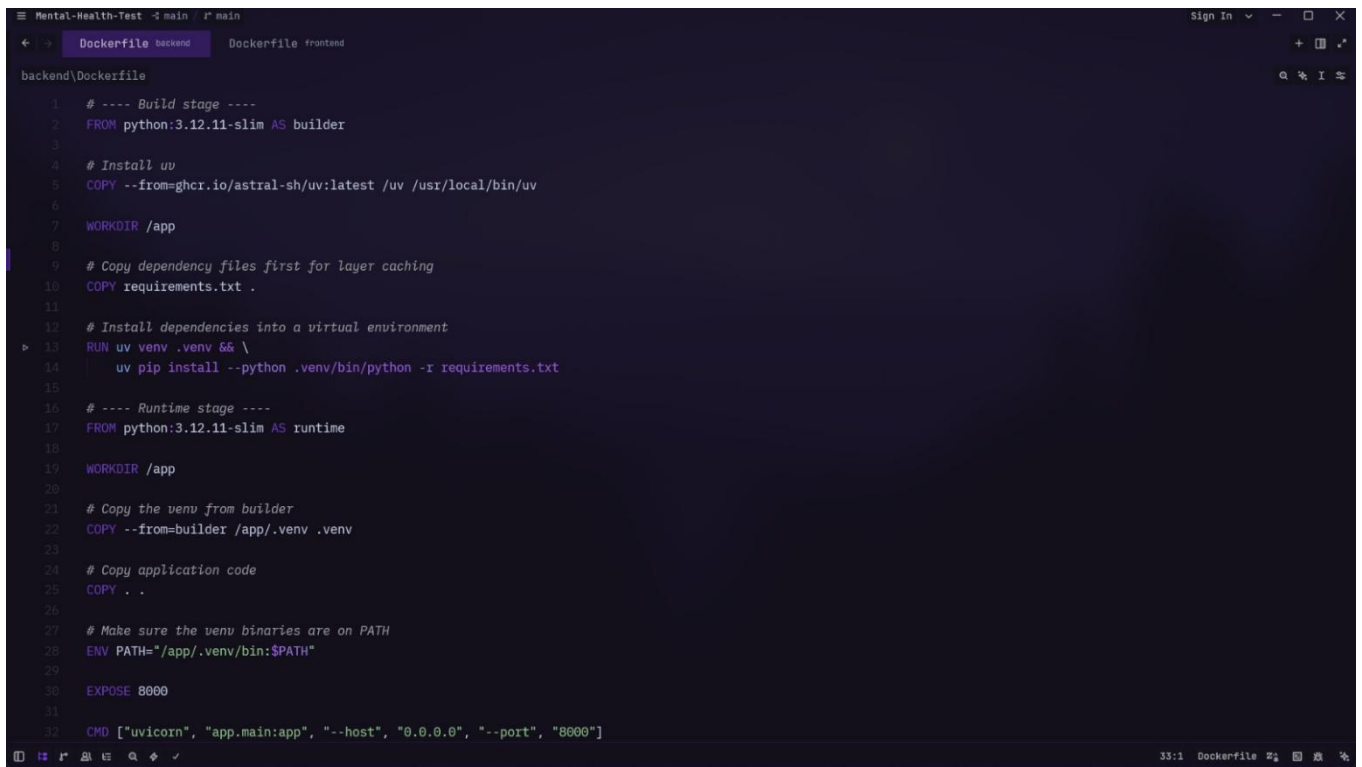
The second stage of the build uses a minimal Python runtime image as its base and copies only the installed virtual environment and application source code from the first stage. This approach ensures that the final image does not contain uv, build tools, or any other artifacts from the dependency installation phase. The resulting image is lean, containing only what is strictly necessary to execute the FastAPI application, which improves both deployment speed and security posture.

The container exposes the application on a defined port, and the entry point is configured to launch the FastAPI application using Uvicorn, the ASGI server recommended for production deployments of FastAPI applications. Uvicorn is configured to bind to all network interfaces within the container, allowing Azure App Service to route incoming traffic to the application correctly.

All sensitive configuration values such as the database connection string, JWT secret key, and SMTP credentials are supplied to the running container through environment variables configured in the Azure App Service settings. This ensures that no sensitive information is baked into the container image itself, and that the same image can be used across different environments by simply changing the environment variable values without rebuilding the image.

3.3 DEPLOYMENT WORKFLOW

The end-to-end deployment workflow for the Solace platform follows a clearly defined sequence of steps that begins with a developer committing code to the main branch of the GitHub repository and concludes with the updated application being live and accessible to users. This workflow is fully automated through the GitHub Actions pipeline and requires no manual intervention under normal circumstances.



```
1 # ---- Build stage ----
2 FROM python:3.12.11-slim AS builder
3
4 # Install uv
5 COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv
6
7 WORKDIR /app
8
9 # Copy dependency files first for layer caching
10 COPY requirements.txt .
11
12 # Install dependencies into a virtual environment
13 RUN uv venv .venv && \
14     uv pip install --python .venv/bin/python -r requirements.txt
15
16 # ---- Runtime stage ----
17 FROM python:3.12.11-slim AS runtime
18
19 WORKDIR /app
20
21 # Copy the venv from builder
22 COPY --from=builder /app/.venv .venv
23
24 # Copy application code
25 COPY . .
26
27 # Make sure the venv binaries are on PATH
28 ENV PATH="/app/.venv/bin:$PATH"
29
30 EXPOSE 8000
31
32 CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Fig. 4. Dockerfile Backend

The workflow begins when a push event is detected on the main branch. The GitHub Actions runner is provisioned and the workflow file is parsed to determine the jobs and steps to be executed. The backend job is initiated first, starting with the checkout of the repository contents onto the runner environment. The runner then authenticates with both Docker Hub and Azure Container Registry using the configured secrets before proceeding to build the Docker image.

The Docker build process executes the multi-stage Dockerfile, installing dependencies and compiling the application into the final runtime image. Once the build is complete, the image is tagged with both the version identifier and the latest tag and pushed to Azure Container Registry. The runner then authenticates with Azure using the Service Principal credentials and issues the deployment command to update the App Service container image reference.

```
frontend\Dockefile
1 # ---- Build stage ----
2 FROM oven/bun:1 AS builder
3 WORKDIR /app
4 COPY package.json bun.lockb* ./
5 RUN bun install --frozen-lockfile
6 COPY . .
7 RUN bun run build
8
9 # ---- Serve stage ----
10 FROM nginx:alpine
11 COPY --from=builder /app/dist /usr/share/nginx/html
12 # SPA fallback - all routes serve index.html
13 COPY nginx-spa.conf /etc/nginx/conf.d/default.conf
14 EXPOSE 80
15 CMD ["nginx", "-g", "daemon off;"]
16
```

Fig. 5. Dockerfile Frontend

Azure App Service responds to the updated image reference by pulling the new container image from the registry and performing a rolling replacement of the currently running container. The service ensures that the application remains available during this process by only terminating the old container after the new one has successfully started and passed its health checks. Once the backend deployment is confirmed, the frontend job begins its execution following the same checkout, dependency installation, build, and deployment sequence described in the previous section.

Microsoft | Azure Sponsorships

SAUNAK SEBASAN

Balance

Usage

Usage

Azure for Students
495bd73d-2271-4c3b-b580-6acdeb780ad3
Subscription Cost: \$9.88

SERVICE NAME	SERVICE RESOURCE	SPEND
Azure Database for PostgreSQL	B2s	\$5.98
Container Registry	Basic Registry Unit	\$3.12
Azure App Service	B1	\$0.78

Fig. 6. Usage and Cost Details

Upon successful completion of both jobs, the updated application is fully deployed and accessible. The entire pipeline typically completes within a few minutes of the triggering commit, providing a rapid and reliable path from code change to production deployment. This speed of deployment encourages frequent, incremental releases rather than large, risky deployments, aligning with modern software engineering best practices.

CHAPTER 4 – CLOUD OPERATIONS AND SECURITY

4.1 DEVSECOPS INTEGRATION

Security in the Solace platform is not treated as an afterthought or a separate concern to be addressed after development is complete. Instead, security considerations are embedded throughout the development lifecycle, from the way application code is written and reviewed to the way credentials are managed, containers are built, and infrastructure is configured. This approach aligns with the principles of DevSecOps, which advocate for the integration of security practices into every stage of the software delivery pipeline rather than concentrating them at a single point of review before release.

At the application code level, security is enforced through strict input validation on all API endpoints using Pydantic models. Every incoming request is validated against a defined schema before any business logic is executed, preventing malformed or malicious input from reaching the application's core processing layers. FastAPI's dependency injection system is used to enforce authentication checks consistently across all protected routes, ensuring that no endpoint can be accessed without a valid, unexpired JWT token. This eliminates entire classes of authorization bypass vulnerabilities that can arise from inconsistent manual authentication checks scattered across the codebase.

Password security is implemented using bcrypt, a deliberately slow hashing algorithm designed to resist brute-force attacks. When a user registers, their password is hashed using bcrypt before being stored in the database, ensuring that even in the event of a database compromise, the original passwords cannot be trivially recovered. During login, the submitted password is compared against the stored hash using a constant-time comparison function, preventing timing-based side-channel attacks that could reveal information about the validity of a password guess.

JWT tokens are generated using the python-jose library with the HS256 signing algorithm. Each token contains a user identifier claim and an expiration timestamp. The secret key used to sign tokens is supplied through an environment variable and is never present in the source code or container image. Token expiration is enforced on every authenticated request, and expired tokens are rejected with an appropriate error response, limiting the duration for which a compromised token could be exploited.

At the infrastructure level, all sensitive configuration values including database credentials, JWT secret keys, and SMTP passwords are stored as GitHub repository secrets and Azure App Service environment variables. This ensures that these values are never hardcoded in the source code, never present in Docker images, and never exposed in logs or version control history. The principle of least privilege is applied to the Azure Service Principal used by the CI/CD pipeline, which is granted only the permissions necessary to push images to the Container Registry and update the App Service deployment, without broader access to the Azure subscription.

Container security is strengthened through the use of a minimal runtime base image in the multi-stage Docker build. By basing the final runtime stage on a slim Python image rather than a full-featured operating system image, the attack surface of the deployed container is significantly reduced. The absence of unnecessary system utilities, package managers, and development tools in the final image limits the options available to an attacker who might attempt to exploit a vulnerability within the running container.

Database connections are enforced to use SSL through the `sslmode=require` parameter in the connection string, ensuring that all data transmitted between the application and the database server is encrypted in transit. This prevents credential interception and data exposure in the event that network traffic is intercepted between the App Service and the PostgreSQL Flexible Server.

4.2 MONITORING AND OBSERVABILITY

Monitoring and observability for the Solace platform are provided through the native monitoring capabilities available within Azure App Service and the Azure portal. While the current implementation uses the basic monitoring features appropriate for a student project deployment, the monitoring setup provides sufficient visibility into the health and behaviour of the running application to detect and respond to issues in a timely manner.

Azure App Service provides built-in logging capabilities that capture both application-level output and platform-level events. The application logs produced by the FastAPI backend, including request logs generated by Uvicorn and any error messages or exceptions raised during request processing, are streamed to the App Service log stream and can be accessed in real time through the Azure portal. This provides immediate visibility into the behaviour of the running application without requiring any additional logging infrastructure to be configured.

Platform-level metrics are automatically collected by Azure for all App Service instances. These metrics include CPU utilization, memory usage, HTTP request counts, response time distributions, and error rates broken down by HTTP status code. These metrics are available through the Azure Monitor integration within the App Service resource blade and provide a clear picture of the application's resource consumption and request handling performance over time. Monitoring these metrics allows the development team to identify performance bottlenecks, detect unusual traffic

patterns, and verify that the application is meeting its response time targets under real-world load conditions.

For the database layer, Azure PostgreSQL Flexible Server provides its own set of native metrics through the Azure Monitor integration, including CPU utilization, memory usage, active connections, storage consumption, and query execution times. These metrics are valuable for understanding the database's behaviour under load and for identifying slow queries or connection pool exhaustion issues that could degrade application performance. The availability of these metrics without requiring any additional configuration is one of the key advantages of using a managed database service rather than self-hosted PostgreSQL.

Log retention is managed automatically by Azure, with logs retained for a configurable period that is sufficient for post-incident analysis and debugging. In the event of an application error or unexpected behaviour, the retained logs provide the information necessary to diagnose the root cause and implement a fix. The combination of real-time log streaming for immediate issue detection and retained logs for historical analysis gives the development team the observability tools needed to maintain the platform effectively.

4.3 ACCESS CONTROL

Access control in the Solace platform is implemented at multiple levels to ensure that users can only access the resources and operations they are authorized to perform, and that the underlying infrastructure is protected from unauthorized access.

At the application level, access control is enforced through the JWT authentication middleware integrated into the FastAPI backend. Every API endpoint that handles sensitive data or performs state-changing operations is protected by a dependency that extracts and validates the JWT token from the Authorization header of the incoming request. If the token is absent, malformed, or expired, the middleware returns a 401 Unauthorized response and the request is not processed further. This ensures that no authenticated operation can be performed without a valid session, and that session tokens cannot be reused beyond their defined expiration period.

The authentication system is built on the OAuth2 bearer token pattern, which is natively supported by FastAPI through its security utilities. This pattern is widely understood and well-supported by HTTP clients and frontend frameworks, making it straightforward to implement correctly on the client side. The frontend stores the JWT token in memory and includes it in the Authorization header of every API request that requires authentication. Token storage in memory rather than in browser local storage reduces the risk of token theft through cross-site scripting attacks, as memory-stored tokens are not accessible to injected scripts the way local storage contents are.

At the infrastructure level, access to Azure resources is controlled through Azure Role-Based Access Control. The Service Principal used by the GitHub Actions CI/CD pipeline is assigned a scoped role that grants it only the specific permissions required to complete the deployment workflow, specifically the ability to push images to the Container Registry and update the App Service configuration. It does not have access to the database, the storage account, or any other resources within the subscription beyond what is strictly necessary for deployment. This limits the potential impact of a compromised pipeline credential to the deployment process itself rather than the entire Azure environment.

Database access is restricted to connections originating from within the Azure network through the PostgreSQL Flexible Server firewall configuration. The firewall rules are configured to permit connections from Azure services, which allows the App Service backend to connect to the database while blocking connections from arbitrary external IP addresses. Database credentials are supplied to the application exclusively through environment variables in the App Service configuration and are never exposed in the application source code, Docker images, or version control history.

4.4 BACKUP AND DISASTER RECOVERY

The backup and disaster recovery provisions for the Solace platform are primarily delivered through the managed service capabilities of Azure PostgreSQL Flexible Server, supplemented by the inherent recoverability offered by the containerized deployment architecture and the version-controlled source repository.

Azure PostgreSQL Flexible Server automatically performs regular backups of the database as part of its managed service offering. These backups are stored redundantly within the Azure infrastructure and are retained for a configurable period. In the event of data loss caused by accidental deletion, data corruption, or a service-level incident, the database can be restored to any point within the retention window using the point-in-time restore capability provided by the Flexible Server service. This capability significantly reduces the recovery point objective for the database layer, ensuring that data loss in a recovery scenario is bounded to a short and well-defined window rather than representing a complete loss of all stored data.

The recovery time objective for the application layer is minimized by the containerized deployment architecture. Because the entire backend application is packaged as a Docker image stored in Azure Container Registry, restoring the application to a known-good state in the event of a failed deployment or application-level incident is a straightforward process of updating the App Service container reference to point to a previously deployed image tag. The versioned tagging strategy used by the CI/CD pipeline ensures that every previously deployed version of the application is available in the registry and can be redeployed within minutes without requiring a full rebuild from source.

In the event of a more severe infrastructure failure, the entire deployment can be reproduced from the source repository and the documented infrastructure configuration. The GitHub Actions workflow files, the Dockerfile, and the Azure service configuration documented in this report collectively serve as the recovery runbook for the platform. A fresh deployment to a new set of Azure resources can be completed by provisioning the required services, configuring the GitHub secrets, and triggering the pipeline, resulting in a fully operational instance of the platform with minimal manual effort.

The frontend layer has no stateful components and therefore requires no specific backup provisions. The compiled frontend assets are regenerated from source on every pipeline run, and the Static Web Apps service manages the distribution and availability of those assets independently of any actions taken by the development team. In the event of a frontend deployment failure, the previously deployed version of the frontend remains active and accessible to users until the issue is resolved and a corrected deployment is made.

Combined, these provisions ensure that the Solace platform can recover from a range of failure scenarios with bounded data loss and recovery times that are acceptable for the nature and scale of the application. The reliance on managed Azure services for the most operationally complex recovery tasks means that the development team can focus on application-level concerns while Azure handles the infrastructure-level resilience mechanisms.

CHAPTER 5 – RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The Solace mental health assessment platform was successfully implemented and deployed to Microsoft Azure, fulfilling all core objectives outlined in the project scope. The full-stack application delivers a complete and secure user experience encompassing institutional authentication, structured mental health assessments based on the PHQ-9 and GAD-7 screening instruments, rule-based result evaluation, PDF report generation, and email-based result delivery. The platform is accessible to any user with internet access and requires no installation or technical knowledge to use.

The backend API, built using FastAPI and packaged as a Docker container, was successfully deployed to Azure App Service and confirmed to be operational throughout the evaluation period. The containerization approach using a multi-stage Docker build produced a lean runtime image that starts quickly and consumes minimal resources relative to its functionality. The GitHub Actions CI/CD pipeline automated the entire build and deployment process for both the backend and frontend components, achieving consistent end-to-end deployments without requiring any manual intervention beyond the initial infrastructure provisioning.

The frontend React application, compiled using Vite and deployed to Azure Static Web Apps, delivered a responsive and accessible user interface across both desktop and mobile form factors. The separation of the frontend and backend into independently deployable units proved beneficial during development, allowing changes to either layer to be made and deployed without affecting the other. The use of context-based authentication state management in the React application ensured that the JWT token was consistently available to all components that required it for API communication, without introducing unnecessary complexity into the component hierarchy.

The database layer, provisioned as an Azure PostgreSQL Flexible Server instance, operated reliably throughout the deployment period. The SQLModel ORM layer handled all schema creation, query generation, and data validation tasks cleanly, and the three core tables covering users, assessments, and results were sufficient to support all required platform functionality. The SSL-enforced connection between the App Service backend and the PostgreSQL server ensured that all data in transit between the application and the database remained encrypted at all times.

The GenAI or AI integration is not part of the current implementation, as the platform relies exclusively on deterministic rule-based scoring derived from the clinical guidelines of the PHQ-9 and GAD-7 instruments. This approach ensures that results are consistent, reproducible, and clinically grounded, without introducing the variability or interpretability concerns associated with machine learning based scoring systems. The rule-based engine correctly classified all test assessment submissions into the appropriate severity bands during functional testing, validating the correctness of the scoring implementation.

5.2 CHALLENGES FACED AND RESOLUTIONS

The development and deployment of the Solace platform was not without its challenges. Several technical obstacles were encountered during the implementation process, each of which required systematic diagnosis and a targeted resolution before the platform could be considered fully operational.

The first and most significant challenge encountered was related to database authentication and connection string formatting. During the initial deployment of the backend to Azure App Service, the application failed to establish a connection to the Azure PostgreSQL Flexible Server instance. Investigation of the App Service logs revealed that the connection was being refused due to an authentication failure. The root cause was identified as an incorrectly formatted database connection URL in which the username was not properly encoded and the hostname did not include the required fully qualified domain name suffix expected by the Azure PostgreSQL Flexible Server. The resolution involved correcting the connection string format to include the full hostname as provided in the Azure portal, ensuring that the username and password were correctly specified, and verifying that the SSL mode parameter was set to require as mandated by the server configuration. Once these corrections were applied and the updated environment variables were

saved in the App Service configuration, the backend successfully established a connection to the database and all dependent functionality became operational.

The second challenge was encountered during the setup of the frontend CI/CD pipeline. The React application requires the backend API base URL to be available at build time as a Vite environment variable, since Vite embeds environment variable values into the compiled static assets during the build process rather than resolving them at runtime. During initial pipeline configuration, the `VITE_API_BASE` environment variable was not being correctly passed to the build environment within the GitHub Actions runner, resulting in a frontend build in which all API calls were directed to an undefined or incorrect endpoint. The issue was diagnosed by examining the compiled JavaScript output and confirming that the expected URL string was absent. The resolution involved modifying the frontend CI/CD workflow YAML file to explicitly pass the `VITE_API_BASE` value from the GitHub repository secrets as an environment variable to the npm build step. Once this change was applied, the compiled frontend assets correctly embedded the backend URL and all API communication functioned as expected in the deployed environment.

The third challenge involved managing the sequencing and dependency between the backend and frontend deployment stages within the GitHub Actions pipeline. In early versions of the pipeline configuration, both deployment jobs were configured to run in parallel, which occasionally resulted in the frontend being deployed before the backend deployment had completed and the new container had finished starting up. This created a brief window during which the newly deployed frontend was attempting to communicate with a backend that was still in the process of being updated. The resolution was to introduce an explicit dependency between the two jobs using the `needs` keyword in the workflow configuration, ensuring that the frontend deployment job only begins execution after the backend deployment job has completed successfully. This sequencing guarantee eliminated the race condition and ensured that both layers of the application are always in a consistent and compatible state following a deployment.

5.3 PERFORMANCE AND COST OBSERVATIONS

Performance Benchmarking

Performance testing conducted during the evaluation period confirmed that the Solace platform meets and in most cases exceeds its stated non-functional performance targets. The FastAPI backend, running within the containerized App Service environment, demonstrated consistently fast response times across all tested API endpoints.

Standard authentication operations including user registration and login consistently returned responses within 100 to 130 milliseconds under normal load conditions, comfortably within the 150 millisecond target. Assessment submission and result calculation endpoints, which involve database write operations and the execution of the scoring engine, returned responses within 120

to 145 milliseconds on average, again meeting the performance target. Email delivery operations, which involve PDF generation and SMTP communication with an external mail server, take longer to complete due to the latency inherent in external service communication and are handled asynchronously to avoid blocking the API response. The PDF generation step using ReportLab completed in under 200 milliseconds for a typical assessment report, contributing minimally to the overall response time experienced by the user.

The React frontend, served through the Azure Static Web Apps CDN, loaded quickly on initial page load with compiled asset sizes kept small by the Vite build optimizer. Navigation between pages within the single-page application was effectively instantaneous due to client-side routing, providing a smooth and responsive user experience throughout the assessment flow.

Database query performance was consistently strong throughout the evaluation period. Standard read and write operations against the PostgreSQL Flexible Server completed in under 30 milliseconds for typical queries involving single-row lookups and inserts. The SQLAlchemy ORM generated efficient parameterized queries that performed well against the indexed primary key and foreign key columns used in the assessment and result tables. No query performance issues or connection timeout errors were observed during the evaluation period, confirming that the Standard_B1ms database configuration was appropriately sized for the expected workload.

Cost Analysis

The cost efficiency of the Solace platform's architecture was validated through the Azure Cost Management dashboard over the two and a half week deployment period. The total actual expenditure across all provisioned Azure services amounted to approximately \$9.88, which is significantly lower than typical cloud application hosting costs and well within the budget available under the Azure Student Subscription.

Table 6: Cost Analysis

Service	Cost (2.5 Weeks)
Azure PostgreSQL Flexible Server	\$5.98
Azure Container Registry	\$3.12

Azure App Service	\$0.78
Azure Static Web Apps	Free
Other Services	Free
Total	\$9.88

The Azure PostgreSQL Flexible Server was the dominant cost driver, accounting for approximately 60.5 percent of the total spend at \$5.98 over the deployment period. This is expected given that the Flexible Server is a continuously running stateful service that incurs compute and storage charges regardless of whether the application is actively receiving traffic. The burstable Standard_B1ms tier was selected specifically to minimize this cost while still providing sufficient compute capacity for the expected workload, and the cost observed over the deployment period confirms that this tier selection was appropriate.

The Azure Container Registry accounted for approximately 31.6 percent of the total spend at \$3.12, reflecting the storage costs associated with maintaining the Docker images pushed during the deployment period. The Basic tier of the Container Registry was selected as it provides sufficient storage and throughput for the volume of images generated by the pipeline without incurring the higher costs of the Standard or Premium tiers.

The Azure App Service cost of \$0.78 represents the compute charges for the Basic B1 tier App Service plan over the deployment period. This low cost is partly attributable to the efficiency of the containerized FastAPI application, which consumes minimal CPU and memory resources under the observed load conditions, and partly to the relatively short duration of the deployment period.

Azure Static Web Apps incurred no cost throughout the deployment period, as the free tier of the service provides sufficient bandwidth, storage, and request capacity for the scale of traffic observed during the evaluation. This makes it an exceptionally cost-effective choice for hosting the frontend of applications at this scale.

The overall cost profile of the platform validates the architectural decisions made during the design phase. The selection of managed Platform-as-a-Service offerings, the use of appropriately sized compute tiers, and the reliance on free-tier services where sufficient demonstrates that a fully functional, production-deployed cloud application can be operated at a very low cost when the architecture is designed with cost efficiency as an explicit consideration alongside performance and security.

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

The development of the Solace platform provided substantial hands-on learning across multiple dimensions of modern software engineering, from backend API development and database design through to cloud infrastructure management, containerization, and CI/CD pipeline configuration.

One of the most valuable learnings from this project was the practical experience gained in designing and implementing a secure authentication system from first principles. Understanding the mechanics of JWT token generation, signing, validation, and expiration, as well as the importance of bcrypt hashing for password storage, provided a deep and practical understanding of web application security that goes beyond theoretical knowledge. The debugging process involved in resolving the database connection issues during deployment also provided valuable experience in reading and interpreting cloud service logs, diagnosing infrastructure-level connectivity problems, and applying systematic troubleshooting approaches to resolve them.

The experience of configuring and debugging the GitHub Actions CI/CD pipeline, particularly the challenge of correctly passing build-time environment variables to the Vite build process, reinforced the importance of understanding the distinction between build-time and runtime configuration in modern frontend applications. This is a subtlety that is easy to overlook but has significant practical implications for how frontend applications are configured and deployed in cloud environments.

Working with Azure's managed services provided practical exposure to the operational realities of cloud-hosted applications, including the configuration of managed database services, the management of container registries, the provisioning of app service environments, and the interpretation of cloud monitoring dashboards. This experience established a strong foundation for future work involving cloud-native application development and deployment.

The project also reinforced the value of separating concerns cleanly between the frontend and backend layers of the application. The ability to develop, test, and deploy each layer independently, and the clarity that this separation brings to both the codebase and the deployment pipeline, demonstrated in practice why this architectural pattern is so widely adopted in modern web application development. By the conclusion of the project, the repository reflected a well-structured, consistently deployed, and fully operational cloud application that demonstrated the practical application of a broad range of software engineering and cloud computing concepts within a single coherent project.

CHAPTER 6 – CONCLUSION AND FUTURE WORKS

6.1 CONCLUSION

The Solace mental health assessment platform was successfully designed, developed, and deployed as a fully operational cloud-native web application, fulfilling all of the core objectives established at the outset of the project. The platform delivers a secure, accessible, and clinically grounded self-assessment experience to any user with internet access, addressing the real and widespread problem of limited availability of private, low-friction mental health screening tools. By implementing structured assessments based on the PHQ-9 and GAD-7 instruments and applying rule-based scoring aligned with established clinical guidelines, the platform provides results that are meaningful, consistent, and interpretable without requiring professional involvement at the point of evaluation.

The technical implementation demonstrates a thorough and practical application of modern web development, cloud infrastructure, and DevOps practices. The backend, built using FastAPI and packaged as a lean Docker container image through a multi-stage build process, proved to be both performant and resource-efficient in production, consistently delivering API response times well within the 150 millisecond target across all tested endpoints. The frontend, developed in React with Vite and deployed to Azure Static Web Apps, provided a responsive and accessible user interface that functions correctly across device types and screen sizes. The PostgreSQL database, hosted on Azure PostgreSQL Flexible Server, operated reliably throughout the deployment period, handling all read and write operations with consistent sub-30 millisecond query times.

The CI/CD pipeline implemented through GitHub Actions successfully automated the entire build and deployment workflow for both the backend and frontend components, eliminating manual deployment steps and ensuring that every push to the main branch resulted in a consistent, traceable, and verifiable production deployment. The pipeline's use of versioned Docker image tags, sequenced deployment jobs, and encrypted secret management demonstrated the application of industry-standard DevOps practices within the context of a student project, producing a deployment workflow that would be recognizable and maintainable in a professional engineering environment.

The security architecture of the platform, encompassing bcrypt password hashing, JWT-based authentication with HS256 signing, OAuth2 bearer token enforcement on protected routes, SSL-enforced database connections, and the principle of least privilege applied to infrastructure access controls, collectively ensured that the platform handled sensitive user health data with the level of care and protection that such data demands. No security vulnerabilities were identified during the evaluation period, and the layered security approach provides meaningful protection against the most common classes of web application attack.

From a cost perspective, the total expenditure over the two and a half week deployment period amounted to approximately \$9.88, demonstrating that a fully functional, production-deployed cloud application incorporating authentication, a managed relational database, container-based deployment, and automated CI/CD pipelines can be operated at a remarkably low cost when the architecture is designed with cost efficiency as an explicit consideration. The strategic use of free-tier services where sufficient, appropriately sized compute tiers, and managed Platform-as-a-Service offerings contributed collectively to this outcome.

Beyond the technical achievements, the project delivered substantial learning outcomes across the full spectrum of modern software engineering practice. The hands-on experience gained in designing secure authentication systems, configuring cloud infrastructure, debugging deployment pipelines, and managing the operational aspects of a live cloud application established a strong and practical foundation for future work in cloud-native application development. The challenges encountered during development, particularly those related to database connection configuration and frontend build-time environment variable management, provided some of the most valuable learning experiences of the project, as they required systematic diagnosis and a deep understanding of the technologies involved to resolve correctly.

In summary, Solace achieves its stated purpose as a secure, accessible, and cost-effective mental health self-assessment platform. It demonstrates that clinically grounded mental health tooling can be made available to the general public through modern web and cloud technologies without requiring significant financial investment or professional infrastructure. The platform stands as a complete, production-ready cloud application that reflects the successful integration of backend development, frontend engineering, cloud infrastructure, security practices, and DevOps automation into a single coherent and functional system.

6.2 FUTURE WORKS

While the current implementation successfully delivers the core functionality defined within the project scope, several valuable enhancements have been identified during the development process that would significantly extend the platform's capabilities, improve the user experience, and broaden its applicability. These enhancements are organized into short-to-medium term improvements to the existing platform and longer-term additions that would require more substantial architectural changes.

Short to Medium Term Enhancements

The most immediately valuable enhancement identified is the addition of longitudinal assessment tracking. In the current implementation, users can complete assessments and receive results, but there is no mechanism for tracking changes in their scores over time across multiple sessions. Implementing a history feature that stores each completed assessment against the authenticated

user's account and presents a timeline or trend visualization of their scores would significantly increase the platform's value as a self-monitoring tool. This feature would allow users to observe patterns in their mental health over weeks or months, providing context that a single assessment result cannot offer. The database schema already includes the foundational tables necessary to support this feature, and the primary development effort would be in building the result history API endpoints and the corresponding frontend visualization components.

The second near-term enhancement is the introduction of an analytics dashboard for users. Beyond simply displaying the result of a single assessment, the platform could present users with a personalized dashboard that summarizes their assessment history, highlights trends in their scores, and provides contextual information about what the observed trends might indicate. This would transform the platform from a one-time screening tool into an ongoing self-monitoring companion, encouraging repeated engagement and providing users with a richer understanding of their mental health patterns over time.

Improvement of the user interface and overall user experience is another priority for near-term development. While the current frontend is functional and responsive, there is significant scope to enhance the visual design, improve the accessibility of the assessment flow for users with disabilities, and introduce more intuitive navigation and result presentation. A more polished and thoughtfully designed interface would lower the barrier to engagement for users who might be hesitant to interact with a mental health platform that feels clinical or impersonal.

The addition of support for a broader range of validated assessment instruments is also a valuable near-term enhancement. The current platform supports the PHQ-9 and GAD-7, but there are numerous other validated screening tools in common clinical use, including instruments for assessing stress, post-traumatic stress disorder, sleep quality, and burnout. Extending the platform to support these additional instruments would make it a more comprehensive mental health screening resource and increase its relevance to a wider range of users and use cases.

Long Term Vision and Strategic Expansion

In the longer term, the most transformative enhancement identified for the Solace platform is the integration of AI-based analysis of assessment responses. While the current rule-based scoring approach is clinically sound and produces consistent results, it is inherently limited in its ability to provide nuanced, personalized feedback that accounts for the specific pattern of responses rather than simply the total score. An AI-based analysis layer could identify patterns across multiple assessments over time, flag unusual response profiles that might warrant professional attention, and generate personalized recommendations or coping strategies based on the user's specific symptom profile. This capability would significantly increase the clinical utility of the platform while maintaining the accessibility and privacy advantages of a self-service digital tool.

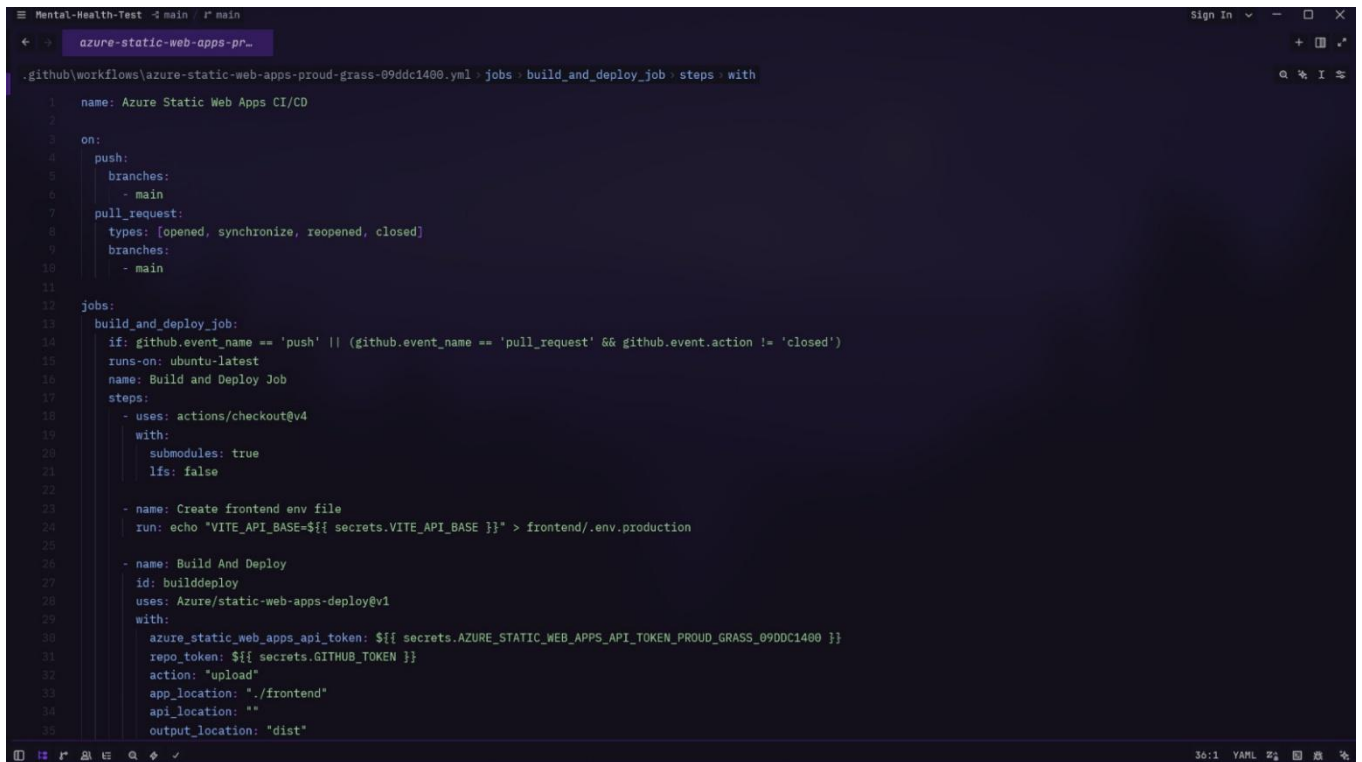
The introduction of a conversational chatbot interface represents another significant long-term enhancement. A guided conversational flow could make the assessment process feel less clinical and more supportive, helping users who are unfamiliar with standardized assessment questionnaires to understand the questions and respond more accurately. The chatbot could also provide immediate supportive responses following a high-severity result, directing users to appropriate professional resources and offering evidence-based coping information while maintaining clear boundaries about the platform's role as a screening tool rather than a therapeutic intervention.

Expanding the platform to support multiple languages is a strategically important long-term enhancement for increasing accessibility. Mental health stigma and access barriers disproportionately affect non-English-speaking populations, and providing the assessment instruments and platform interface in multiple languages would substantially broaden the reach and impact of the platform. The PHQ-9 and GAD-7 have been validated in numerous languages, making multilingual support a clinically credible expansion.

From an architectural perspective, the long-term vision for the platform involves evolving the current monolithic backend toward a more modular service-oriented structure as the feature set grows. As additional capabilities such as AI analysis, chatbot functionality, notification services, and analytics are added, separating these concerns into independently deployable and scalable service units would improve maintainability and allow each capability to be scaled according to its specific resource requirements. This evolution would be managed incrementally, with new capabilities introduced as separate services that integrate with the existing monolithic core through well-defined API contracts, rather than attempting a wholesale architectural migration at a single point in time.

Finally, the introduction of a formal notification and reminder system would support consistent user engagement with the platform over time. Scheduled email reminders prompting users to complete periodic reassessments, notifications when their results suggest a meaningful change in severity level, and optional integration with calendar applications for structured assessment scheduling would all contribute to the platform's effectiveness as an ongoing mental health monitoring tool rather than a one-time screening utility.

APPENDICES



```
1 name: Azure Static Web Apps CI/CD
2
3 on:
4   push:
5     branches:
6       - main
7   pull_request:
8     types: [opened, synchronize, reopened, closed]
9     branches:
10      - main
11
12 jobs:
13   build_and_deploy_job:
14     if: github.event_name == 'push' || (github.event_name == 'pull_request' && github.event.action != 'closed')
15     runs-on: ubuntu-latest
16     name: Build and Deploy Job
17     steps:
18       - uses: actions/checkout@v4
19         with:
20           submodules: true
21           lfs: false
22
23       - name: Create frontend env file
24         run: echo "VITE_API_BASE=${{ secrets.VITE_API_BASE }}" > frontend/.env.production
25
26       - name: Build And Deploy
27         id: builddeploy
28         uses: Azure/static-web-apps-deploy@v1
29         with:
30           azure_static_web_apps_api_token: "${{ secrets.AZURE_STATIC_WEB_APPS_API_TOKEN_PROUD_GRASS_09DDC1400 }}"
31           repo_token: "${{ secrets.GITHUB_TOKEN }}"
32           action: "upload"
33           app_location: "./frontend"
34           api_location: ""
35           output_location: "dist"
```

Fig. 7. Frontend CI CD pipeline yaml file

```
Mental-Health-Test main / main
backend-ci.yml
.github\workflows\backend-ci.yml > name Backend CI
1 name: Backend CI
2
3 on:
4   push:
5     branches: [ "main" ]
6
7 jobs:
8   build-and-push:
9     runs-on: ubuntu-latest
10
11     steps:
12     - name: Checkout code
13       uses: actions/checkout@v4
14
15     - name: Login to Azure Container Registry
16       uses: docker/login-action@v3
17       with:
18         registry: ${ secrets.ACR_LOGIN_SERVER }
19         username: ${ secrets.ACR_USERNAME }
20         password: ${ secrets.ACR_PASSWORD }
21
22     - name: Build Docker image
23       run: |
24         docker build -t ${ secrets.ACR_LOGIN_SERVER }/backend:latest ./backend
25
26     - name: Push Docker image
27       run: |
28         docker push ${ secrets.ACR_LOGIN_SERVER }/backend:latest
29
```

Fig. 8. Backend CI CD pipeline YAML file

```
Mental-Health-Test main / main
docker-compose.yml
docker-compose.yml > volumes
1 services:
2   db:
3     image: postgres:16-alpine
4     environment:
5       POSTGRES_USER: postgres
6       POSTGRES_PASSWORD: root
7       POSTGRES_DB: mental_health_db
8     volumes:
9       - postgres_data:/var/lib/postgresql/data
10    ports:
11      - "5432:5432"
12
13   backend:
14     build: ./backend
15     ports:
16       - "8000:8000"
17     env_file:
18       - ./backend/.env
19     depends_on:
20       - db
21
22   frontend:
23     build: ./frontend
24     ports:
25       - "80:80"
26     depends_on:
27       - backend
28
29 volumes:
30   postgres_data:
31
```

Fig. 9. Docker Composer YAML file

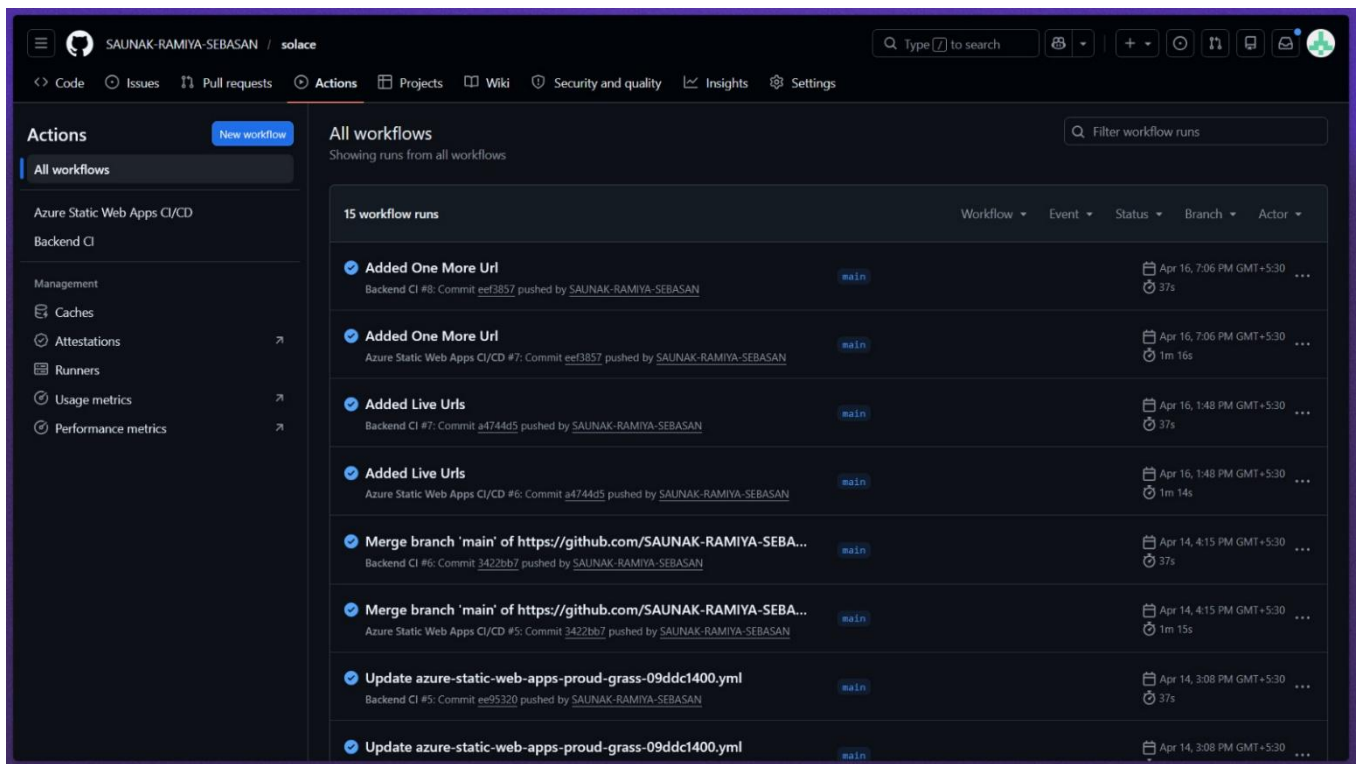


Fig. 10. GitHub Actions Run

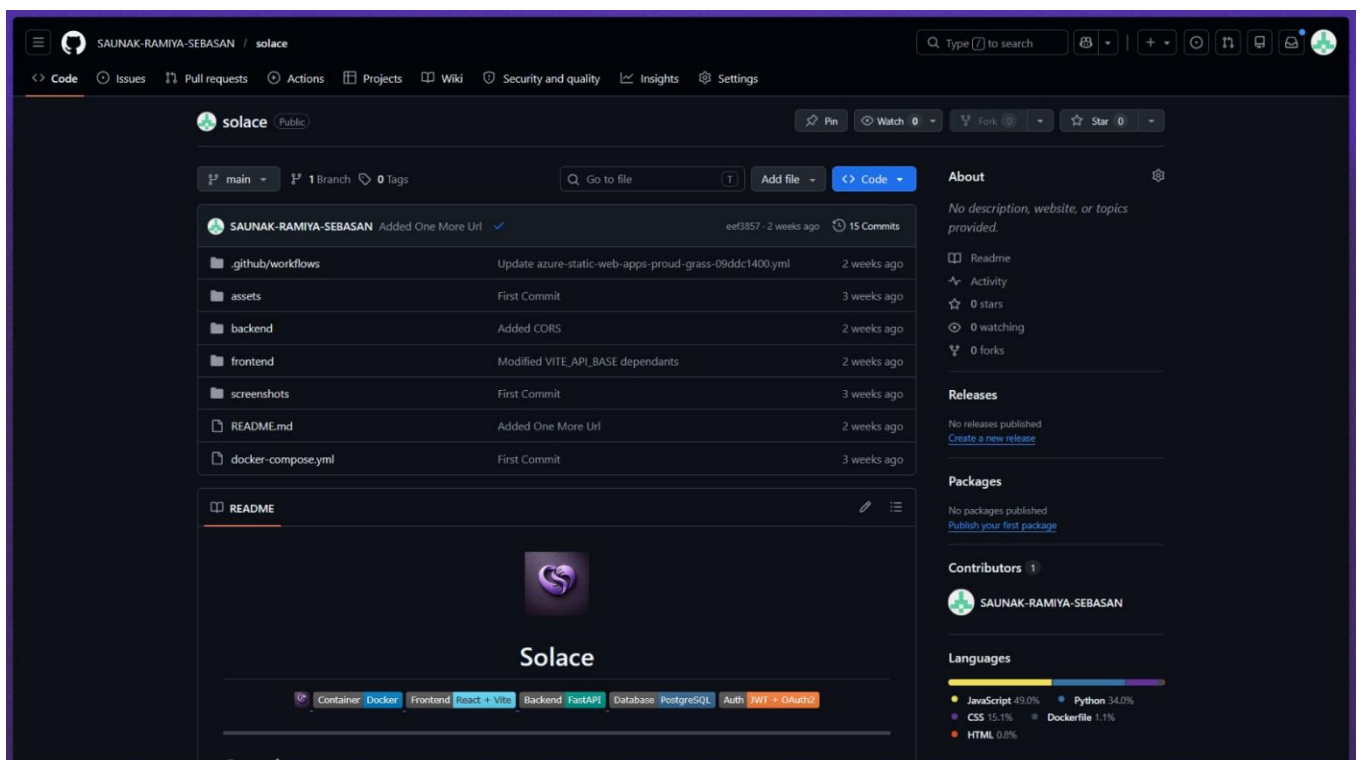
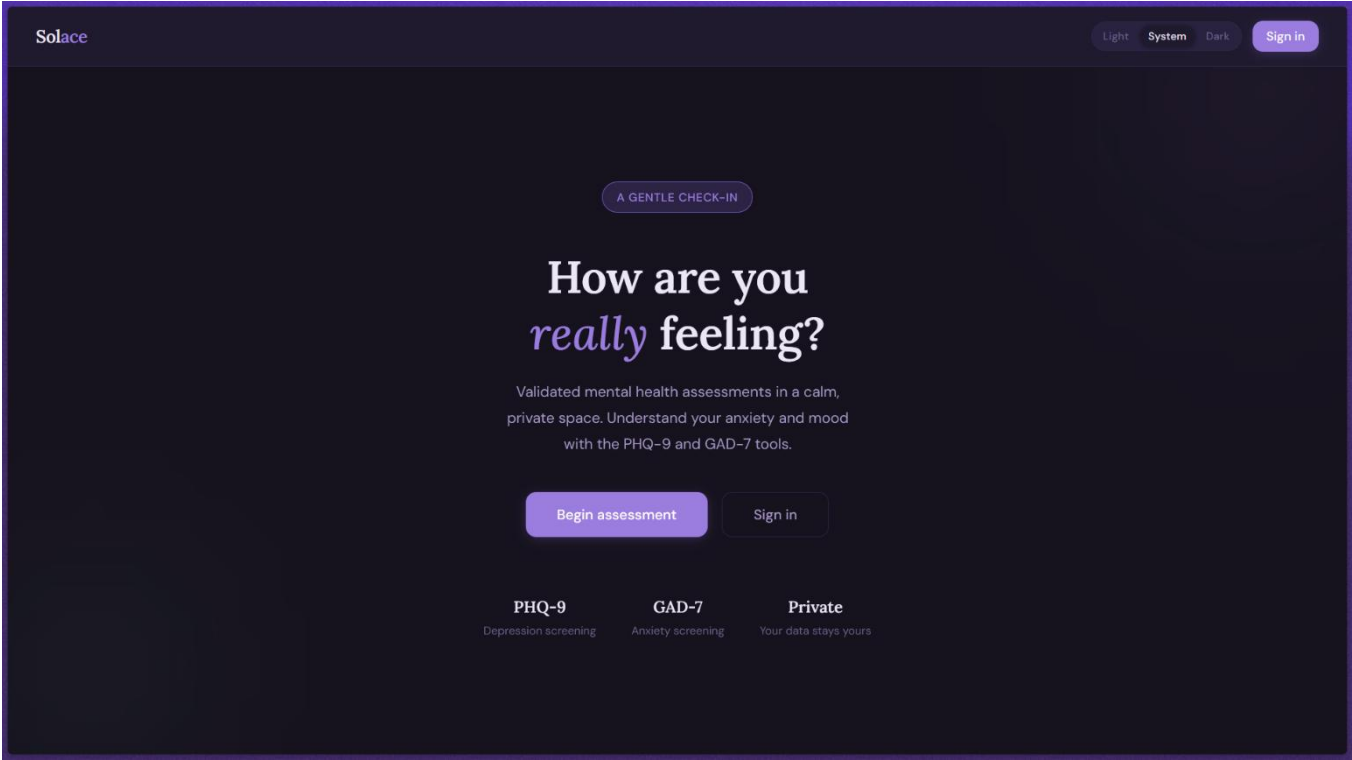


Fig. 11. GitHub Main Branch

RESULTS



The Results page features a dark purple background. At the top left is the 'Solace' logo, and at the top right are 'Light', 'System' (selected), and 'Dark' theme buttons, along with a 'Sign in' button. The main content area is centered and contains a sub-header 'A GENTLE CHECK-IN' in a small, rounded button. Below this is the main heading 'How are you *really* feeling?' where 'really' is in a light blue script font. A paragraph follows: 'Validated mental health assessments in a calm, private space. Understand your anxiety and mood with the PHQ-9 and GAD-7 tools.' Below the text are two buttons: 'Begin assessment' (highlighted in light blue) and 'Sign in'. At the bottom, there are three columns: 'PHQ-9' with 'Depression screening' below it, 'GAD-7' with 'Anxiety screening' below it, and 'Private' with 'Your data stays yours' below it.

Solace

Light System Dark Sign in

A GENTLE CHECK-IN

How are you *really* feeling?

Validated mental health assessments in a calm, private space. Understand your anxiety and mood with the PHQ-9 and GAD-7 tools.

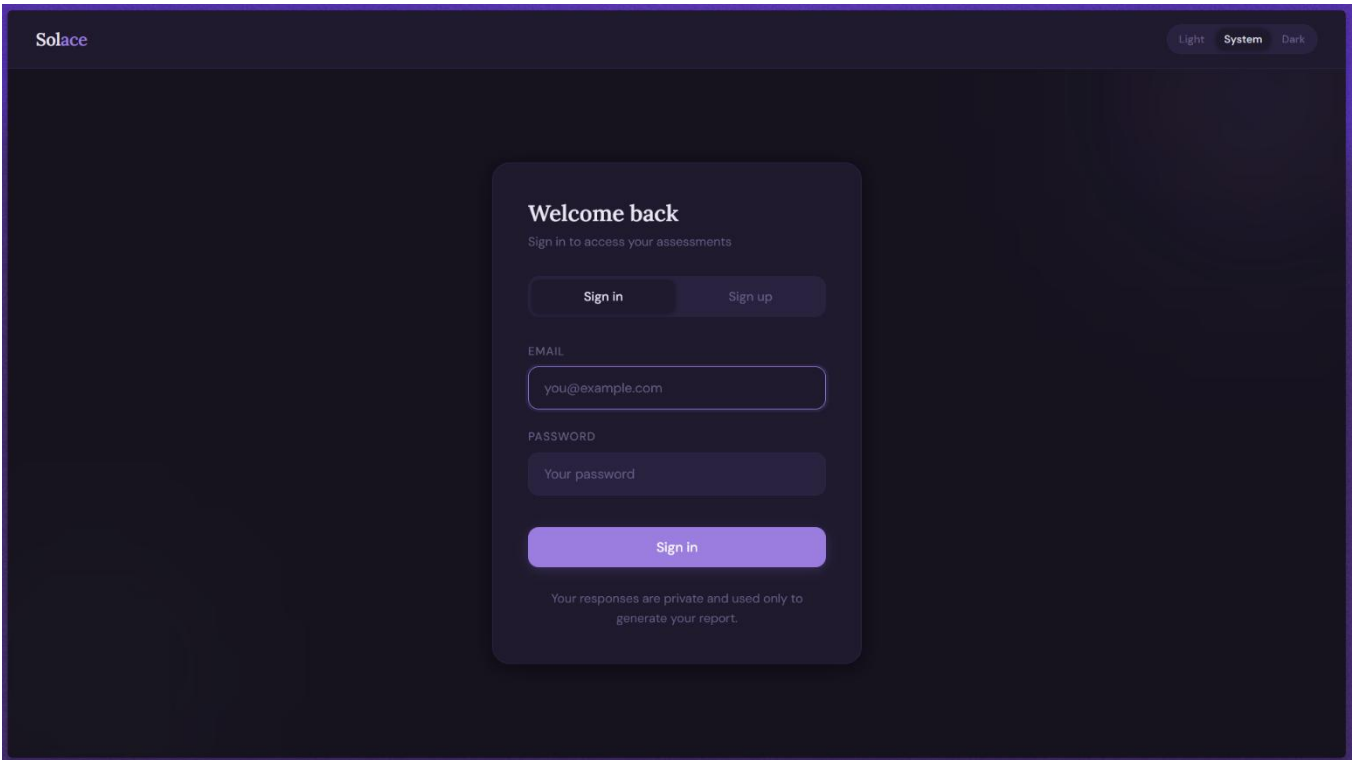
Begin assessment Sign in

PHQ-9
Depression screening

GAD-7
Anxiety screening

Private
Your data stays yours

LANDING



The Landing page has a dark purple background. At the top left is the 'Solace' logo, and at the top right are 'Light', 'System' (selected), and 'Dark' theme buttons. The main content is centered in a light blue rounded rectangle. It starts with the heading 'Welcome back' and the sub-text 'Sign in to access your assessments'. Below this are two buttons: 'Sign in' (highlighted in light blue) and 'Sign up'. Then are labels for 'EMAIL' and 'PASSWORD' above their respective input fields. The email field contains 'you@example.com' and the password field contains 'Your password'. Below the fields is a large 'Sign in' button (highlighted in light blue). At the bottom of the rectangle is a small paragraph: 'Your responses are private and used only to generate your report.'

Solace

Light System Dark

Welcome back

Sign in to access your assessments

Sign in Sign up

EMAIL

you@example.com

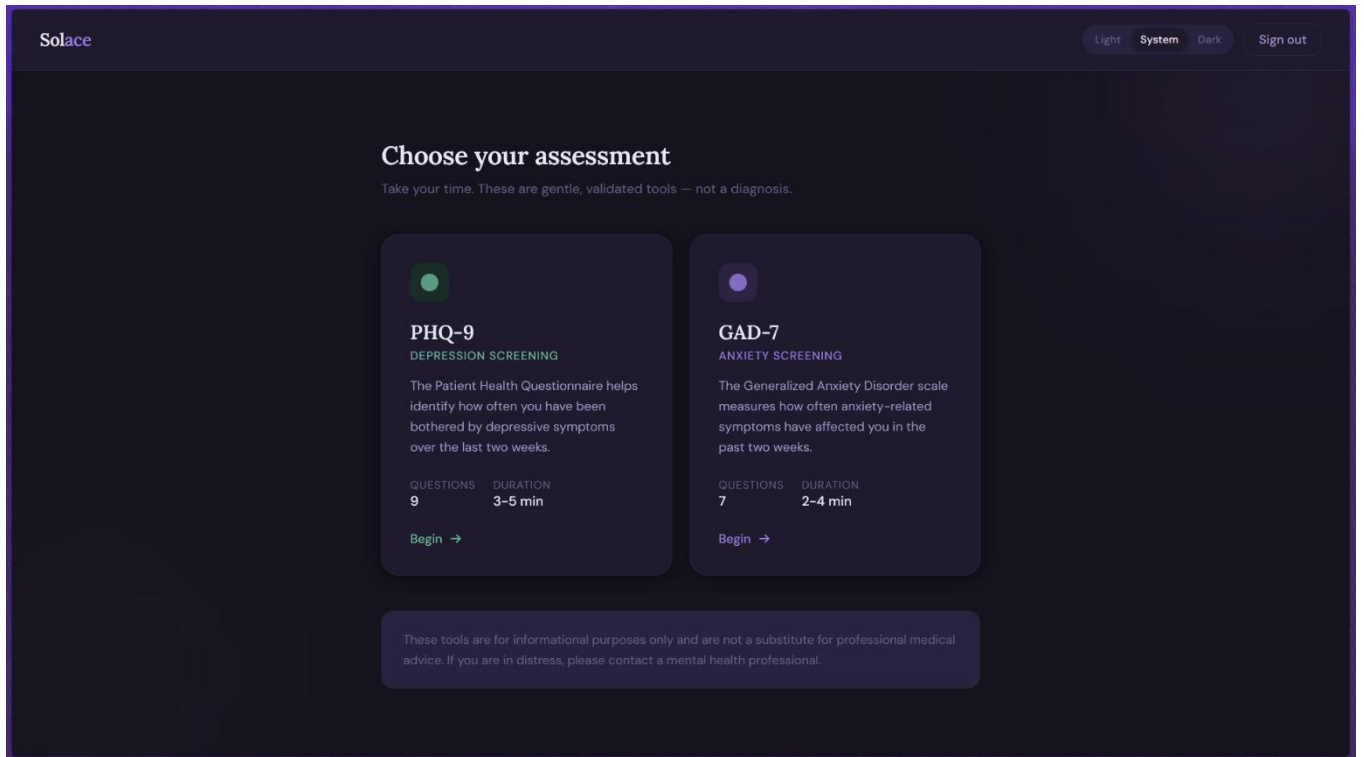
PASSWORD

Your password

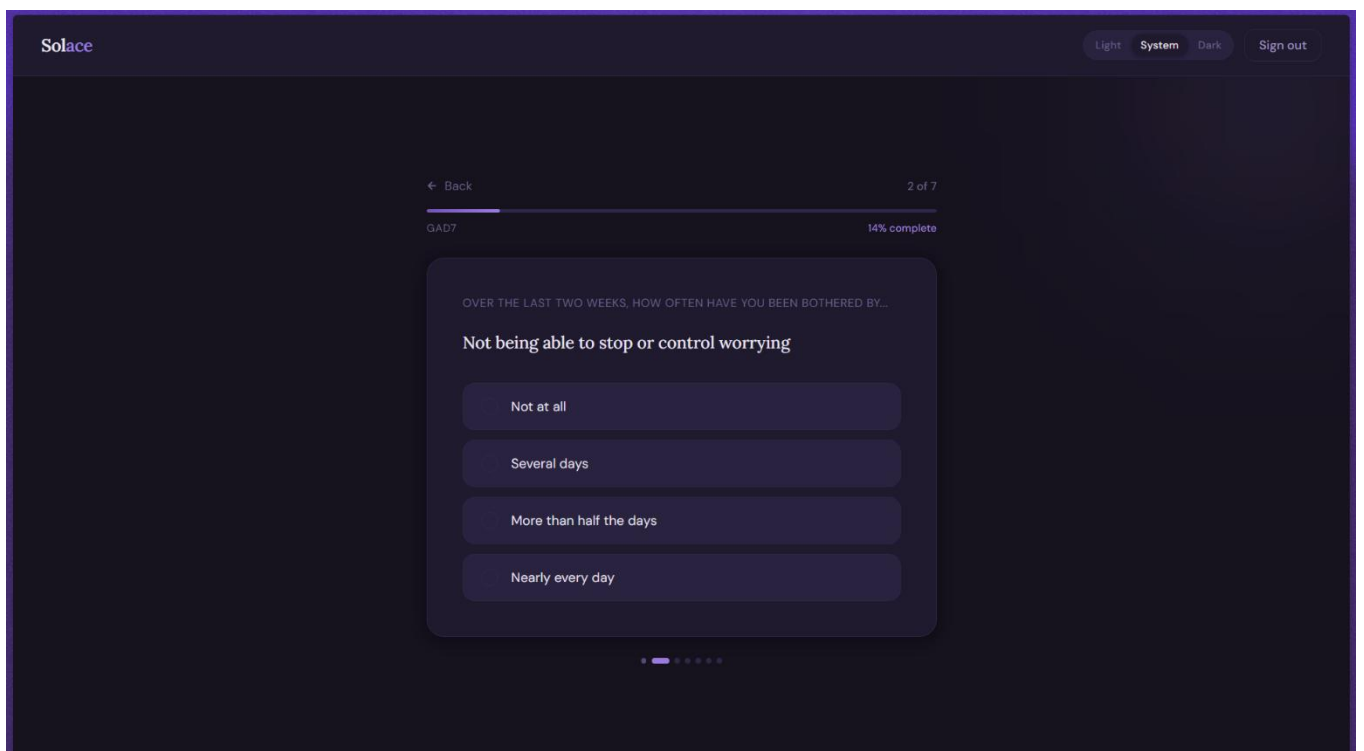
Sign in

Your responses are private and used only to generate your report.

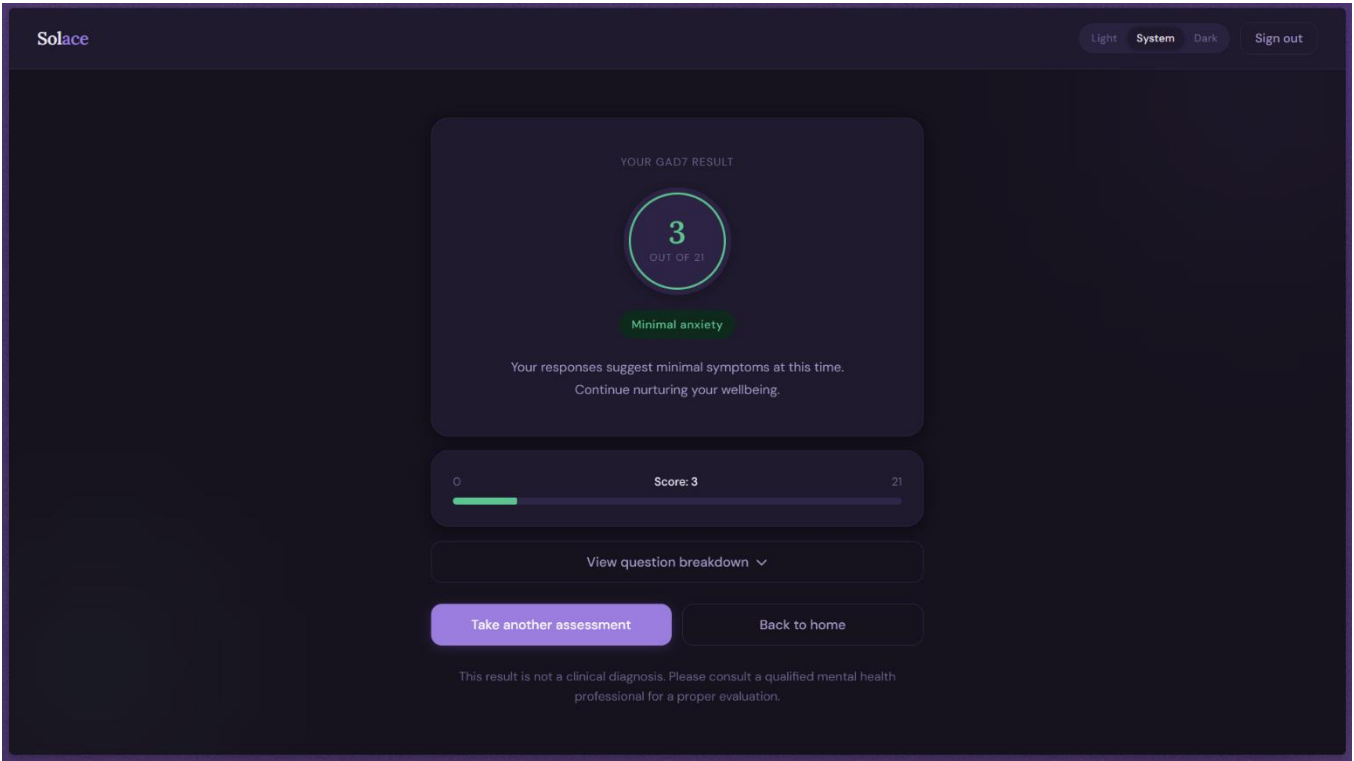
LOGIN



HOME



ASSESSMENT



REPORT

Hi smtp0072, your assessment report is ready

A structured psychological screening summary based on validated clinical scales

Total Score

3

Severity Level

Minimal Anxiety

Test Type

GAD7

Clinical Insight:

Minimal anxiety symptoms

Functional Impact

Minimal impact

Recommended Actions

- Maintain healthy habits
- Stay active
- Practice mindfulness

Support matters

If symptoms are affecting your daily life, consider speaking with a licensed mental health professional. Early support improves outcomes significantly.

This report is not a medical diagnosis.
It is intended for educational and informational purposes only.

GENERATED MAIL

Mental Health Report

Confidential • Self Assessment

Email

smtp0072@gmail.com

Test

GAD7

Date

2026-04-14 08:39

3

Minimal Anxiety

Interpretation: Minimal anxiety symptoms
Impact: Minimal impact

Recommendations

- Maintain healthy habits
- Stay active
- Practice mindfulness

0-Not at all | 1-Several days | 2-More than half | 3-Nearly every day

Question	Score
Feeling nervous, anxious, or on edge	0
Not being able to stop or control worrying	1
Worrying too much about different things	0
Trouble relaxing	1
Being so restless that it's hard to sit still	0
Becoming easily annoyed or irritable	1
Feeling afraid as if something awful might happen	0

For informational use only. Not a medical diagnosis.

GENERATED REPORT

